version 2107301230 4
version 2112311200 4
version 2203021531 4
version 2204051645 4
version 2207141040 4
version 2208101410 4
version 2211090955 4
version 2211161440 4
version 2301241350 4
version 2303271010 4
version 2304121540 4
version 2312131530 5
version 2509041522 5
1. COMMUNICATION PROTOCOL 6
1.1. Message format from the software application to the CU 6
1.2. Message format from the CU to the software application 7
1.2.1. Acknowledgement response 7
1.2.2. Message response 8
1.3. Short messages for testing the status of the CU 8
2. DESCRIPTION OF THE COMMANDS 9
2.1. Format and presentation of commands 9
2.2. General commands 10
2.2.1. Command: 20h / SP - Status 10
2.2.2. Command: 21h / ! - Version 11
2.2.3. Command: 24h / $ – Clear display 11
2.2.4. Command: 25h / % – Display text line 1 11
2.2.5. Command: 26h / & – Display text line 2 12
2.2.6. Command: 27h / '– Display text lines 1 and 2 12
2.2.7. Command: 28h / (– Display date and time 12
2.3. Configuration commands 13
2.3.1. Command: 40h / @ – Set Manufacturing number 13
2.3.1.1. Command: 41h / A– Set PIN number, Option 1 13
2.3.1.2. Command: 41h / A – Confirm fiscalization, Option 2 13
2.3.1.3. Command: 41h / A – Change SD card, Option 3 13
2.3.2. Command: 42h / B – Change VAT rates 13
2.3.3. Command: 3Fh / ? – Software Reset 15
2.3.4. Command: 48h / H – Program date and time 15
2.3.5. Command: 4Fh / O – Program HS code at a given position, Option Z 15
2.3.6. Command: 4Fh / O –Append the new HS code, Option Z, W 15
2.3.7. Command: 4Fh / O – Erase of HS codes, option z, D 16
2.3.8. Command: 7Ch / | – Start device update 16
2.4. Data reading commands 17
2.4.1. Command: 60h / ' – Read CU numbers 17
2.4.2. Command: 62h / b – Read VAT rates 17
2.4.3. Command: 68h / h – Read date and time 17
2.4.4. Command: 22h / ” – Read Diagnostics 18
2.4.5. Command: 4Fh / O – Read HS code from a given position, Option Z, R 18
2.4.6. Command: 4Fh / O – Read the number of HS codes, option z, R 18
2.4.7. Command: 5Ah / Z – Read info from the last server communication 19
2.4.8. Command: 5Ah / Z – Program service contract password 19
2.4.9. Command: 5Ah / Z – Activation with the expiration date of the contract 19
2.4.10. Command: 5Ah / Z – Program service contract warning message 20
2.4.11. Command: 5Ah / Z – Read service contract expiry date 20
2.4.12. Command: 5Ah / Z – Read service contract warning message 20
2.4.13. Command: 5Dh /] – Read CU Invoce number 21
2.5. Receipt operations commands 22
2.5.1. Command: 30h / 0 –Open Fiscal sales receipt 22
2.5.2. Command: 30h / 0 – Open Fiscal Invoice sale receipt 23
2.5.3. Command: 30h / 0 – Open Fiscal Invoice Credit Note receipt 24
2.5.4. Command: 30h / 0 – Open Fiscal Invoice Debit Note receipt 25
2.5.5. Command: 31h / 1 – Sell of article 26
2.5.5. Command: 31h / 1 – Sell of article with defined HS code 26
2.5.6. Command: 2Ch / , – Open receipt, sell of article and close receipt 26
2.5.7. Command: 2Ch / , – Open Fiscal Invoice, sell of article and close receipt 28
2.5.6. Command: 33h / 3 – Subtotal 29
2.5.7. Command: 38h / 8 – Close fiscal receipt 29
2.5.8. Command: 39h / 9 – Cancel fiscal receipt 29
2.5.9. Command: 3Ah / : – Inform about issued receipt duplicate 29
2.6. Commands for reading the data in CU's registers 31
2.6.1. Command: 6Dh / m – Read amounts by VAT Class 31
2.6.2. Command: 6Dh / m – Read EOD amounts, option ‘d’ 31
2.6.3. Command: 6Dh / m – Read first EOD date 31
2.6.4. Command: 6Dh / m – Read EOD amounts by date 32
2.6.5. Command: 71h / q – Read receipt numbers 32

2.6.6. Command: 72h / r – Read information about the current opened receipt....33
2.7. Reports commands....34
2.7.1. Command: 7Ch / | – Inform for daily fiscal Z report....34
2.7.2. Command: 7Ch / | –Store Electronic Journal report from date to date....34
2.7.3. Command: 7Ch / | –Store Electronic Journal report from datetime to datetime....34
2.7.4. Command: 7Ch / | –Store Electronic Journal report by CU Invoice Number....35
2.7.5. Command: 7Ch / | –Store Electronic Journal report by PIN of buyer....35
2.7.6. Command: 7Ch / | –Store Electronic Journal report by Trader System Invoice Number....35
2.7.7. Command: 7Ch / | –Store Electronic Journal report from beginning to end....36
2.7.8. Command: 7Ch / | – Read Electronic Journal report from date to date....36
2.7.9. Command: 7Ch / | – Read Electronic Journal report from date to date in JSON format....36
2.7.10. Command: 7Ch / | – Read Electronic Journal report from datetime to datetime....37
2.7.11. Command: 7Ch / | – Read Electronic Journal report from datetime to datetime in JSON format....37
2.7.12. Command: 7Ch / | – Read Electronic Journal report by CU Invoice Number....37
2.7.13. Command: 7Ch / | – Read Electronic Journal report by CU Invoice Number in JSON format....38
2.7.14. Command: 7Ch / | – Read Electronic Journal report by PIN of buyer....38
2.7.15. Command: 7Ch / | – Read Electronic Journal report by PIN of buyer in JSON format....38
2.7.16. Command: 7Ch / | – Read Electronic Journal report by Trader System Invoice Number....39
2.7.17. Command: 7Ch / | – Read Electronic Journal report by Trader System Invoice Number in JSON format....39
2.7.18. Command: 7Ch / | – Read Electronic Journal report from beginning to end....39
2.7.19. Command: 7Ch / | – Read Electronic Journal report from beginning to end in JSON format....40
2.7.20. Command: 7Ch / | – Read/Store Copy of Invoice receipt....40
2.8. Settings LAN/WiFi/GPRS commands....41
2.8.1. Command: 4Eh / N – Read Device modules support by Firmware....41
2.8.2. Command: 4Eh / N – Read Device modules support....42
2.8.3. Command: 4Eh / N – Read TCP password....42
2.8.4. Command: 4Eh / N – Read TCP auto start....43
2.8.5. Command: 4Eh / N – Read Device TCP addresses....43
2.8.6. Command: 4Eh / N – Read Device TCP MAC address....43
2.8.7. Command: 4Eh / N – Read Device NTP address....44
2.8.8. Command: 4Eh / N – Read TCP DHCP enabled....45
2.8.9. Command: 4Eh / N – Scan for available WiFi networks....45
2.8.10. Command: 4Eh / N – Read TCP WiFi network name....45
2.8.11. Command: 4Eh / N – Read TCP WiFi password....46
2.8.12. Command: 4Eh / N – Read TCP used communication module, LAN or WiFi....46
2.8.13. Command: 4Eh / N – Read TCP idle timeout....46
2.8.14. Command: 4Eh / N – Read GPRS APN....47
2.8.15. Command: 4Eh / N – Read GPRS password....47
2.8.16. Command: 4Eh / N – Read GPRS username....47
2.8.17. Command: 4Eh / N – Read HTTPS server address....48
2.8.18. Command: 4Eh / N – Read Invoice threshold count....48
2.8.19. Command: 4Eh / N – Read time threshold minutes....49
2.8.20. Command: 4Eh / N – Read Server Communication Module....49
2.8.21. Command: 4Eh / N – Read total message count....49
2.8.22. Command: 4Eh / N – Read specific message....50
2.8.23. Command: 4Eh / N – Read GPRS APN autehntication type....50
2.8.24. Command: 4Eh / N – Read TCP port number....51
2.8.25. Command: 4Eh / N – Read proxy server address....51
2.8.26. Command: 4Eh / N – Read server reading timeout....51
2.8.27. Command: 4Eh / N – Set TCP password....51
2.8.28. Command: 4Eh / N – Set TCP auto start....52
2.8.29. Command: 4Eh / N – Set Device TCP addresses....53
2.8.30. Command: 4Eh / N – Set Device TCP MAC address....53
2.8.31. Command: 4Eh / N – Set Device NTP address....53
2.8.32. Command: 4Eh / N – Set TCP DHCP enabled....54
2.8.33. Command: 4Eh / N – Set TCP WiFi network name....54
2.8.34. Command: 4Eh / N – Set TCP WiFi password....54
2.8.35. Command: 4Eh / N – Set TCP active communication module, LAN or WiFi....55
2.8.36. Command: 4Eh / N – Set idle timeout....55
2.8.37. Command: 4Eh / N – Set GPRS APN....55
2.8.38. Command: 4Eh / N – Set GPRS password....56
2.8.39. Command: 4Eh / N – Set GPRS username....56
2.8.40. Command: 4Eh / N – Set HTTPS server address....56
2.8.41. Command: 4Eh / N – Set Invoice threshold count....56
2.8.42. Command: 4Eh / N – Set time threshold minutes....57
2.8.43. Command: 4Eh / N – Set server communication module....57
2.8.44. Command: 4Eh / N – Set GPRS APN autehntication type....57
2.8.45. Command: 4Eh / N – Set TCP port number....58
2.8.46. Command: 4Eh / N – Set proxy server address....58
2.8.47. Command: 4Eh / N – Set server reading timeout....58
2.8.48. Command: 4Eh / N – Save network settings....58
2.8.49. Command: 4Eh / N – Start device LAN test....58
2.8.50. Command: 4Eh / N – Start device WiFi test....59
2.8.51. Command: 4Eh / N – Start device GPRS test....59

3. SOFTWARE APPLICATION REQUIREMENTS 60
3.1. Rules for using the commands 60
3.2. Sample sale transaction of CU 60
4. AUXILARY GS PROTOCOL AND ESC COMMANDS 61

VERSION 2107301230
Changed lengh of *RelatedInvoiceNum* [19]
VERSION 2112311200
Added Print/Store Copy of Invoice receipt**:** ReadOrStoreInvoiceCopy(CUInvoiceNum)
VERSION 2203021531
Adding command with zfpdef: ReadInfoFromLastServerCommunication() for reading the info from last communication with server
VERSION 2204051645
Adding command with zfpdef: ReadEODAmounts*()* for reading the EOD turnover sum and VAT
VERSION 2207141040
Removed duplication of command with zfpdef: ScanWiFiNetworks()
VERSION 2208101410
New status ST5 bit 6 added “Update is available”.
Adding command with zfpdef: *StartDeviceUpdate()* which starts a device update procedure if a new update is available.
Adding commands with zfpdefs: ReadTcpPortNumber(), SetTcpPortNumber() to read and save the TCP port of the device.
VERSION 2211090955
Added option “Response Code” in the command with zfpdef:
ReadInfoFromLastServerCommunication()
VERSION 2211161440
Added commands for reading and programming proxy server address with zfpdefs:
ReadProxyServerAddress(), ProgramProxyServerAddress()

VERSION 2301241350
Added new commands for reading and store the Electronic Journal report with zfpdefs:
StoreEJByDateTime,
StoreEJByCUInvoiceNum,
StoreEJByPINofBuyer,
StoreEJByTraderSystemInvNum,
ReadEJByDateTime,
ReadEJByDateTime_JSON,
ReadEJByCUInvoiceNum,
ReadEJByCUInvoiceNum_JSON,
ReadEJByPINofBuyer,
ReadEJByPINofBuyer_JSON,
ReadEJByTraderSystemInvNum,
ReadEJByTraderSystemInvNum_JSON

VERSION 2303271010
Added option to select EJ report format - Brief or Detailed for commands with zfpdefs:
StoreEJByDateTime,
StoreEJByCUInvoiceNum,
StoreEJByPINofBuyer,
StoreEJByTraderSystemInvNum,
ReadEJByDateTime,
ReadEJByDateTime_JSON,
ReadEJByCUInvoiceNum,
ReadEJByCUInvoiceNum_JSON,
ReadEJByPINofBuyer,
ReadEJByPINofBuyer_JSON,
ReadEJByTraderSystemInvNum,
ReadEJByTraderSystemInvNum_JSON
VERSION 2304121540
ChangeSDcard

VERSION 2312131530

Added commands for reading and programming the timeout when reading from server.
zfpdefs: Read_ServerReadingTimeout(), ProgramServerReadingTimeout()

VERSION 2509041522

Added new commands for reading and programming the server contract options.

ProgramServiceContractPassword*(Old_Passowrd,New_Passowrd)*
ProgramExpirationServiceContract*(Passowrd,Activation,ExpiryDate)*
ProgContractWarningMessages(*Password,Line1,Line2,Line3*)
ReadServiceContractExpiryDate()
ReadContractWarningMessage()

Added new command for request for original invoice number in case of open receipt rejection due to duplicate TRADER SYSTEM INVOICE NUMBER and response received
ReadCUinvoiceNumber()

1. COMMUNICATION PROTOCOL

The type of the protocol is Master / Slave. The communication session is always initiated by the Application Software. CU caries out the commands send by the software application and provides a feedback depending on the result. CU sends back an „Acknowledgement response" or „message response". All messages of the protocol are either packed or single-byte. CU supports communication standard RS232, USB, BT, TCP (WiFi, LAN).

Serial port adjustment parameters:

Speed: 115200 bit/s (or 19200, 38400, 57600 and 9600 if such is set for the CU)
8 bits word
No parity
1 stop bit

1.1. MESSAGE FORMAT FROM THE SOFTWARE APPLICATION TO THE CU

All messages except those described in section 4., sent to the CU by the PC have the following structure:

<STX><LEN><NBL><CMD><DATA…DATA><CS><CS><ETX>

The table below contains description of the field enclosed between the symbols **<** and **>**:

Field	No. of bytes	Value
STX	1	***Message start*** – always **02h**
LEN	1	***Message length*** (number of bytes including LEN, NBL, CMD, DATA) increased by **20h** i.e. a number in the 20h - 9Fh range
NBL	1	***Message number*** increased by **20h** i.e. a number in the 20h - 9Fh range
CMD	1	***Command*** - a number in the 20h - 7Fh range(see the description of commands)
DATA.. DATA	0 - 3902	***Additional data*** – a group of data fields separated with the symbols **';'**, giving additional information needed for execution of the command (see the description of commands)
CS CS	2	***Checksum***, compiled as follows: 1) Operation XOR of all bytes from LEN to DATA inclusive = 0 .. FFh 2) Conversion of 2 bytes by adding 30h, for example: B5h -> 3Bh 35h
ETX	1	***End of message*** – always **0Ah** (LF)

The texts data of the message is sent as ASCII text with code table cp1251 (Windows 1251).

1.2. MESSAGE FORMAT FROM THE CU TO THE SOFTWARE APPLICATION

There are several types of response depending on the message received.

1.2.1. ACKNOWLEDGEMENT RESPONSE

Positive acknowledgement – when package format is correct. It is sent when the command is acknowledged as well as when it is rejected (errors in the data sent (field <DATA…DATA>) or the command cannot be executed or the command is illegal depending on the current status of the CU indicated by the two status bytes). It is a package message with the following format:

<ACK><NBL><STE><STE><CS><CS><ETX>

Fields description:

Field	No bytes	Value
ACK	1	06h
NBL	1	***No. of message*** = NBL of message related to receipt
STE STE	2	***2 error status-bytes***. A two-digit ASCII number. (see Table Errors)
CS CS	2	***Checksum***, compiled as follows: 1) Operation XOR on NBL STE and STE = 00h .. FFh 2) Conversion of 2 bytes by adding 30h, for example:B5h -> 3Bh 35h
ETX	1	0Ah (LF)

The two status-bytes are a two-digit ASCII number, in which the first digit provides information about the error in the CU, and the second one – about a command error.

Table Errors:

Byte value	*FD errors*	*Byte value*	*Command errors*
30	OK	**30**	OK
31	Not used	**31**	Invalid command
32	Registers overflow	**32**	Illegal command
33	Clock failure or incorrect date&time	**33**	Z daily report is not zero
34	Opened fiscal receipt	**34**	Syntax error
35	Not used	**35**	Input registers overflow
36	Not used	**36**	Zero input registers
37	Not used	**37**	Unavailable transaction for correction
38	Not used	**38**	Insufficient amount on hand
39	Incorrect password	**39**	Conflict in tax group
3a	Not used	**3a**	Invoice number not found!
3b	24hours block – missing Z report	**3b**	Wrong client PIN!
3c	Not used	**3c**	Missing defined item!
3d	Interrupt power supply in fiscal receipt (one time until status is read)	**3d**	Item quantity too large!
3e	Overflow EJ	**3e**	Item amount too large!
3f	Insufficient conditions	**3f**	Amount exceeds original invoice
		40	HS code is not valid

A two-digit number is compiled depending on the type of error.

Example: Error 32 – Illegal command due to clock failure

Negative acknowledgement – It is sent when the package format is incorrect. It is 1-byte **NACK = 15h** without checksum.

Repetition request – It is sent when the CU is busy executing the preceding command. It is 1 byte **RETRY = 0Eh** without checksum.

1.2.2. MESSAGE RESPONSE

It has the format of the packed message sent by the SA to the CU (see 3.1.) but is returned by the CU to the SA and contains information – response to the query (see description of commands).

1.3. SHORT MESSAGES FOR TESTING THE STATUS OF THE CU

The exchange protocol includes two unpacked single-byte codes for testing the status of the CU, which can quickly determine the status of the device. The two codes and their meaning are shown in the table below:

<table>
<tr><th>Query SA</th><th colspan="2">Response CU</th><th>Meaning</th></tr>
<tr><td>04</td><td colspan="2">04</td><td>CU is on</td></tr>
<tr><td rowspan="5">09</td><td>40</td><td rowspan="2">Bit.0</td><td>CU is READY</td></tr>
<tr><td>41</td><td>CU is busy</td></tr>
<tr><td>50</td><td colspan="2">CU is waiting for password (LAN or WiFi connection only)</td></tr>
<tr><td>60</td><td colspan="2">CU is already busy with another connection (LAN or WiFi connection only)</td></tr>
<tr><td>70</td><td colspan="2">Wrong password (LAN or WiFi connection only)</td></tr>
</table>

The format of the commands is described in art. 2. **DESCRIPTION OF THE COMMANDS OF FISCAL DEVICE**

2. DESCRIPTION OF THE COMMANDS

2.1. Format and presentation of commands

All commands are described and presented using the following terms and symbols:

Key terms:

Command – the value of the CMD field of the message sent by the software application and in the message response of the the CU.

input – structure of the fields included in the DATA field of the message sent by the software application.

output – for each command it may be one of the following:

- Acknowledgement response.
- Structure of the fields included in the DATA field of the message response sent by the CU.

Input data – description of the contents of the “input“ fields.

Output data – description of the contents of the “output“ fields.

Key symbols:

' '	– compulsory symbol
“ ”	– compulsory DateTime format
< >	– compulsory data field
<;>	– field separator
[ ]	– field length
{ }	– non-compulsory data field
zfpdef:	- function name in the generated source code and file server file name

General rules:

Format of the price/value field – from 1 to 10 symbols, a floating decimal point number, preceded by +, - or SPACE.

Examples: -12.34 +56.7 8

Format of the quantity field – from 1 to 10 symbols, a floating decimal point number, up to three digits after the decimal point.

Examples: 1.234 56.78 9

Format of the rate (percentage) field – from 2 to 7 symbols, a floating decimal point number, up to two digits after the decimal point, preceded by the percent symbol - %.

Examples: -12.34% +5.67% 8.9% 10%

Payment Number 0 corresponds to the main payment – IN CASH, payment Number 11 corresponds to the special currency payment - VAT account.

2.2. General commands

These are commands for the general functions of the CU, related to obtaining diagnostic information and to direct access to some of the functions of the device (paper feeding, paper cutting, and display visualization).

2.2.1. Command: 20h / SP - Status

input: n. a.
output: <*StatusBytes[6]*>
FPR operation: Provides detailed 6-byte information about the current status of the CU.
Input data : n. a.
Output data :

N byte	*N bit*	*status flag*
ST0	0	reserved
	1	Power down in opened fiscal receipt
	2	reserved
	3	DateTime not set
	4	DateTime wrong
	5	RAM reset
	6	Hardware clock error
	7	1

ST1	0	reserved
	1	Reports registers Overflow
	2	reserved
	3	reserved
	4	reserved
	5	reserved
	6	reserved
	7	1

ST2	0	reserved
	1	Opened Fiscal Receipt
	2	Receipt Invoice Type
	3	reserved
	4	reserved
	5	SD card near full
	6	SD card full
	7	1

ST3	0	reserved
	1	reserved
	2	reserved
	3	reserved
	4	reserved
	5	CU fiscalized
	6	CU produced
	7	1

ST4	0	Paired with TIMS
	1	Unsent receipts
	2	Service contract nearly expired
	3	reserved
	4	reserved
	5	reserved
	6	reserved
	7	1

ST5	0	No Sec.IC
	1	No certificates
	2	**Service jumper**
	3	Expired service contract
	4	Missing SD card
	5	Wrong SD card
	6	Update is available
	7	1

zfpdef: ReadStatus()

2.2.2. Command: 21h / ! - Version

input: n. a.
output: <*Version[30]*>
FPR operation: Provides information about the device version.

Input data : n. a.
Output data :

Version	Up to 30 symbols for Version name and Check sum

zfpdef: ReadVersion()

2.2.3. Command: 24h / $ – Clear display

input: n. a.
output: ACK
FPR operation: Clears the external display.

Input data : n. a.
Output data : n. a.
zfpdef: ClearDisplay()

2.2.4. Command: 25h / % – Display text line 1

input: <*Text[16]*>

output: ACK
FPR operation: Shows a 20-symbols text in the upper external display line.

Input data :

Text 16 symbols text

Output data: n. a.

zfpdef: DisplayTextLine1(Text)

2.2.5. Command: 26h / & – Display text line 2

input: <*Text[16]*>
output: ACK
FPR operation: Shows a 16-symbols text in the lower external display line.

Input data :

Text 16 symbols text

Output data: n. a.

zfpdef: DisplayTextLine2(Text)

2.2.6. Command: 27h / '– Display text lines 1 and 2

input: <*Text[32]*>
output: ACK
FPR operation: Shows a 16-symbols text in the first line and last 16-symbols text in the second line of the external display lines.

Input data :

Text 32 symbols text

Output data: n. a.

zfpdef: DisplayTextLines1and2(Text)

2.2.7. Command: 28h / (– Display date and time

input: n. a.
output: ACK
FPR operation: Shows the current date and time on the external display.

Input data : n. a.

Output data : n. a.

zfpdef: DisplayDateTime()

2.3. Configuration commands

These are commands requiring data recording in the CU. Password access is required.

2.3.1. Command: 40h / @ – Set Manufacturing number

input: *<Password[6]> <;> <SerialNum[20]>*
output: ACK
FPR operation: Stores the Manufacturing number into the operative memory.

Input data :

Password	6-symbols string
SerialNum	(Serial Number) 20 symbols Manufacturing number

Output data: n. a.
zfpdef: SetSerialNum(Password, SerialNum)

2.3.1.1. Command: 41h / A– Set PIN number, Option 1

input: *<Password[6]> <;> <'1'> <;> <PINnum[11]>*
output: ACK
FPR operation: Stores PIN number in operative memory.

Input data :

Password	6-symbols string
'1'	One symbol is compulsory 1
PINnum	11 symbols for PIN registration number

Output data: n. a.
zfpdef: SetPINnumber(Password, *PINnum*)

2.3.1.2. Command: 41h / A – Confirm fiscalization, Option 2

input: *<Password[6]> <;> <'2'>*
output: ACK
FPR operation: Confirm PIN number.

Input data :

Password	6-symbols string
'2'	One symbol for option

Output data: n. a.
zfpdef: ConfirmFiscalization(Password)

2.3.1.3. Command: 41h / A – Change SD card, Option 3

input: *<Password[6]> <;> <'3'> <;> <SerialNum[20]><;><PINnum[11]>*
output: ACK
FPR operation: Change SD card.

Input data :

Password	6-symbols string
'3'	One symbol for option
SerialNum	(Serial Number) 20 symbols Manufacturing number for verification with original
PINnum	11 symbols for PIN registration number for verification with original

Output data: n. a.
zfpdef: ChangeSDcard(Password, SerialNum, PINnum)

2.3.2. Command: 42h / B – Change VAT rates

input: *<Password[6]> <;> <VATrateA[1..6]> <;> <VATrateB[1..6]> <;> <VATrateC[1..6]> <;> <VATrateD[1..6]> <;><VATrateE[1..6]>*
output: ACK
FPR operation: Stores a block containing the values of the VAT rates into the CU

Input data :

Password	6-symbols string

VATrateA	(VAT rate A) Value of VAT rate A from 2 to 6 symbols with format ##.##
VATrateB	(VAT rate B) Value of VAT rate B from 2 to 6 symbols with format ##.##
VATrateC	(VAT rate C) Value of VAT rate C from 2 to 6 symbols with format ##.##
VATrateD	(VAT rate D) Value of VAT rate D from 2 to 6 symbols with format ##.##
VATrateE	(VAT rate E) Value of VAT rate E from 2 to 6 symbols with format ##.##

Output data: n. a.

zfpdef: ProgVATrates(Password, VAT*rateA*, VAT*rateB,* VAT*rateC,* VAT*rateD, VATrateE*)

2.3.3. Command: 3Fh / ? – Software Reset

input: *<Password[6]>*
output: ACK
FPR operation: Restore default parameters of the device.

Input data :

Password	6-symbols string

Output data: n. a.
zfpdef: SoftwareReset(Password)

2.3.4. Command: 48h / H – Program date and time

input: *<DateTime "DD-MM-YY HH:MM">*
output: ACK
FPR operation: Sets the date and time and current values.

Input data :

DateTime	Date Time parameter in format: *DD-MM-YY HH:MM*

Output data : n. a.
zfpdef: *SetDateTime(DateTime)*

2.3.5. Command: 4Fh / O – Program HS code at a given position, Option Z

input: *<Option['Z']> <;><OptionW['W']><;><HS_Number[4]> <;> <HS_Code[10]> <;> <HS_Name[20]><;><OptionTaxable[1]> <;> <MesureUnit[3]> <;> < VAT_Rate[2..6]>*
output: ACK
FPR operation: Programs HS code at a given position (HS number in order).

Input data :

Option	1 symbol with value 'Z'
OptionW	1 symbol with value 'W'
HS_Number	4 symbols for HS number in order in format ####
HS_Code	10 symbols for HS code
HS_Name	20 symbols for name of HS group
OptionTaxable	1 symbol for parameter: - '1' - Exempted - '0' - Taxable
MesureUnit	3 symbols for mesure unit of item's code
VAT_Rate	(VAT rate) Value of VAT rate from 2 to 5 symbols with format ##.##

Output data: n. a.
zfpdef: ProgHScode(*HS_Number,HS_Code, NS_Name, VAT_rate,ParExempt,MesuareUnit)*

2.3.6. Command: 4Fh / O –Append the new HS code, Option Z, W

input: *<Option['Z']> <;>< OptionW['W']><;><reserved['0000']> <;> <HS_Code[10]> <;> <HS_Name[20]><;><OptionTaxable[1]><;><MesureUnit[3]><;><VAT_rate[2..6]>*
output: ACK
FPR operation: Programs the customer DB for special customer receipt issuing.

Input data :

Option	1 symbol with value 'Z'
OptionW	1 symbol with value 'W'
reserved	4 symbols with value '0000'
HS_Code	10 symbols for HS code
HS_Name	20 symbols for name of HS group
OptionTaxable	1 symbol for parameter: - '1' - Exempted - '0' - Taxable
MesureUnit	3 symbols for mesure unit of item's code
VAT_rate	(VAT rate) Value of VAT rate from 2 to 5 symbols with format ##.##

Output data: n. a.
zfpdef: AddNewHScode(*HS_Number,HS_Code, NS_Name, VAT_rate,ParExempt,MesuareUnit)*

2.3.7. Command: 4Fh / O – Erase of HS codes, option z, D

input: *<Option['z']><;><OptionR[‘D’]><;><Password[6]>*
output: ACK
FPR operation: Erase HS codes.

Input data :

Option	1 symbol with value 'z'
OptionD	1 symbol with value ‘D’
Password	6 symbols for password

Output data: n. a.
zfpdef: EraseHScodes*()*

2.3.8. Command: 7Ch / | – Start device update

Input: *<’U’>*
Output: *ACK*
FPR Operation: Start a device update procedure if a new update is available.

Input data:

‘U’	1 symbol with value 'U'

Output data: n.a.
zfpdef: *StartDeviceUpdate()*

2.4. Data reading commands

Set of commands for receiving information from the CU about programmed values as well as additional information.

2.4.1. Command: 60h / ' – Read CU numbers

input: n. a.

output: *<SerialNumber[20]><;><PINnumber[11]>-*

FPR operation: Provides information about the manufacturing number of the CU and PIN number.

Input data : n. a.

Output data :

SerialNumber	20 symbols for individual number of the CU
PINnumber	11 symbols for pin number

zfpdef: ReadCUnumbers ()

2.4.2. Command: 62h / b – Read VAT rates

input: n. a.

output: *<VATrateA[1..7]> <;> <VATrateB[1..7]> <;> <VATrateC[1..7]> <;> <VATrateD[1..7]> <;> <VATrateE[1..7]>*

FPR operation: Provides information about the current VAT rates (the last value stored in FM).

Input data : n. a.

Output data :

VATrateA	(VAT rate A) Up to 7 symbols for VATrates of VAT class A in format ##.##%
VATrateB	(VAT rate B) Up to 7 symbols for VATrates of VAT class B in format ##.##%
VATrateC	(VAT rate C) Up to 7 symbols for VATrates of VAT class C in format ##.##%
VATrateD	(VAT rate D) Up to 7 symbols for VATrates of VAT class D in format ##.##%
VATrateE	(VAT rate E) Up to 7 symbols for VATrates of VAT class E in format ##.##%

Zfpdef: *ReadVATrates()*

2.4.3. Command: 68h / h – Read date and time

input: n. a.

output:*<DateTime "DD-MM-YYYY HH:MM">*

FPR operation: Provides information about the current date and time.

Input data : n. a.

Output data :

DateTime	Date Time parameter in format: DD-MM-YY [Space] hh:mm

Zfpdef: *ReadDateTime()*

2.4.4. Command: 22h / ” – Read Diagnostics

input: ***n.a.***

output: *<DeviceType[1]> <;> <SDIdxPos[10]> <;> <LastInvoiceCUNum[19]> <;> <LastInvoiceDate[6]> <;> <LastEODDate[6]> <;> <InvoicesSent[4]>*

FPR operation: Provides information about documents sending functions .

Input data : n.a.

Output data:

DeviceType	1 symbol for device type: - '1' – A Type - '2' – B Type
SDIdxPos	10 symbols for current SD index position of last sent receipt
LastInvoiceCUNum	19 symbols for number of last invoice according the CU
LastInvoiceDate	6 symbols for last invoice date in the DDMMYY format
LastEODDate	6 symbols for last sent EOD in the DDMMYY format
InvoicesSent	4 symbold for number of invoices sent for the current day

zfpdef:*ReadDiagnostics()*

2.4.5. Command: 4Fh / O – Read HS code from a given position, Option Z, R

input: *<Option['Z']> <;><OptionR[‘R’]><;><HS_Number[4]>*

output: *<Option['Z']> <;><OptionR[‘R’]><;><HS_Number[4]> <;> <HS_Code[10]> <;> <HS_Name[20]> <;><OptionTaxable[1]> <;> <MesureUnit[3]> <;> < VAT_Rate[2..6]>*

FPR operation: Programs HS code at a given position (HS number in order).

Input data :

Option	1 symbol with value 'Z'
OptionR	1 symbol with value ‘R’
HS_Number	4 symbols for HS number in order in format ####

Output data:

Option	1 symbol with value 'Z'
OptionR	1 symbol with value ‘R’
HS_Number	4 symbols for HS number in order in format ####
HS_Code	10 symbols for HS code
HS_Name	20 symbols for name of HS group
OptionTaxable	1 symbol for parameter: - ‘1’ - Exempted - ‘0’ - Taxable
MesureUnit	3 symbols for mesure unit of item’s code
VAT_Rate	(VAT rate) Value of VAT rate from 2 to 5 symbols with format ##.##

zfpdef: ReadHScode(*HS_Number)*

2.4.6. Command: 4Fh / O – Read the number of HS codes, option z, R

input: *<Option['z']> <;><OptionR[‘R’]>*

output: *<Option['z']> <;><OptionR[‘R’]><;>< HScodesNumber [4]>*

FPR operation: Read the number of HS codes.

Input data :

Option	1 symbol with value 'z'
OptionR	1 symbol with value ‘R’

Output data:

Option	1 symbol with value 'z'
OptionR	1 symbol with value ‘R’
HScodesNumber	4 symbols for HS codes number in format ####

zfpdef: ReadHScodeNumber()

2.4.7. Command: 5Ah / Z – Read info from the last server communication

input: *<Option['S']><;> <ServerResponse[1]><;><TransactionType[1]>*
output: *<Option['S']><;> <ServerResponse[1]><;><TransactionType[1]><;><Message[200]>*
FPR operation: Provide information from the last communication with the server.
Input data :

Option	1 symbol with value 'S'
ServerResponse	1 symbol with value - 'R' – At send receipt - 'Z' – At send EOD
TransactionType	1 symbol with value - 'c' – Error Code - 'm' – Error Message - 's' – Status - 'e' – Exception Message - 'r' – Response Code

Output data:

Option	1 symbol with value 'S'
ServerResponse	1 symbol with value - 'R' – At send receipt - 'Z' – At send EOD
TransactionType	1 symbol with value - 'c' – Error Code - 'm' – Error Message - 's' – Status - 'e' – Exception Message - 'r' – Response Code
Message	Up to 200 symbols for the message from the server

zfpdef: ReadInfoFromLastServerCommunication()

2.4.8. Command: 5Ah / Z – Program service contract password

input: <Option['P']> *<;><Old_Password[5]> <;><New_Password[5]>*
output: ACK
FPR operation: Program service contract password
Input data :

Option	*1 symbol with value 'P'*
Old_Password	*5 symbols for **previous** password*
New_Password	*5 symbols for new password*

Output data: n. a.
zfpdef: ProgramServiceContractPassword*(Old_Passowrd,New_Passowrd)*

2.4.9. Command: 5Ah / Z – Activation with the expiration date of the contract

input: <Option['B']> *<;><Password[5]> <;><Activation[1]>*
<;><ExpiryDate "DDMMYY">
output: ACK
FPR operation: Program expiration date of the service contract
Input data :

Option	1 symbol with value 'B'
Password	*5 symbols for password*
Activation	1 symbol with value: - '0'- No - '1'- Yes
ExpiryDate	*6 symbols for date in format DDMMYY*

Output data: n. a.
zfpdef: ProgramExpirationServiceContract*(Passowrd,Activation,ExpiryDate)*

2.4.10. Command: 5Ah / Z – Program service contract warning message

input: *<Option['M']> <;><Password[6]> <;> <Line1[TextLength]> <;><Line2[TextLength]> <;> <Line3[TextLength]>*

output: ACK

FPR operation: Program the service contract expired warning message text.

Input data :

Option	1 symbol with value 'M'
Password	6 symbols for service password
Line1	*TextLength* symbols for warning message of line 1
Line2	*TextLength* symbols for warning message of line 2
Line3	*TextLength* symbols for warning message of line 3

Output data: n. a.

zfpdef: ProgContractWarningMessages(*Password,Line1,Line2,Line3*)

2.4.11. Command: 5Ah / Z – Read service contract expiry date

input: <Option['D']>

output: <Option['D']><;><*Activation[1]*><;> <*ExpiryDate "DDMMYYYY"*>

FPR operation: Read the service contract expiry date

Input data :

Option	1 symbol with value 'D'

Output data:

Option	1 symbol with value 'D'
Activation	1 symbol with value: - '0'- No - '1'- Yes
ExpiryDate	8 symbols for date in the format 'DDMMYYYY'

zfpdef: ReadServiceContractExpiryDate()

2.4.12. Command: 5Ah / Z – Read service contract warning message

input: *<Option['m']>*

output: *<Option['m']><;> <Line1[TextLength]> <;><Line2[TextLength]> <;> <Line3[TextLengt*

FPR operation: Read the service contract warning message text.

Input data :

Option	1 symbol with value 'm'

Output data:

Option	1 symbol with value 'm'
Line1	*TextLength* symbols for warning message of line 1
Line2	*TextLength* symbols for warning message of line 2
Line3	*TextLength* symbols for warning message of line 3

zfpdef: ReadContractWarningMessage()

2.4.13. Command: 5Dh /] – Read CU Invoce number

input: <*Option['I']*>
output: <*Option['I']*><;> <*CU_InvoiceNumber[19]*>
FPR operation: Request for original invoice number in case of open receipt rejection due to duplicate TRADER SYSTEM INVOICE NUMBER and response received.

Input data :

Option	1 symbol with value 'I'

Output data:

Option	1 symbol with value 'I'
CU_InvoiceNumber	19 symbols for CU Invoice number

zfpdef: ReadCUinvoiceNumber()

2.5. RECEIPT OPERATIONS COMMANDS

These commands are used for CU sales registration.

2.5.1. Command: 30h / 0 –Open Fiscal sales receipt

input:<reserved*[‘1’]*> <;> <*reserved[‘ 0’]*> <;> <*ReceiptFormat[1]*> <;> <*ReceiptType[‘0’]*><;><*TraderSystemInvNum[15]*>

output: ACK

FPR operation: Opens a fiscal receipt assigned to the specified operator number and operator password, parameters for receipt format and VAT type.

Input data :

reserved	1 symbol with value ‘1’
reserved	1 symbol with value ‘ 0’
ReceiptFormat	1 symbol with value: - '1' - Detailed - '0' – Brief
ReceiptType	1 symbol with value ‘0’
TraderSystemInvNum	15 symbols for trader system invoice number

Output data: n. a.

zfpdef: *OpenReceipt(ReceiptType ReceiptFormat,TraderSystemInvNum)*

2.5.2. Command: 30h / 0 – Open Fiscal Invoice sale receipt

input: <reserved*['1']*> *<;>* *<reserved[' 0']>* *<;>* *<reserved['0']>* *<;>* <InvoiceType*['1']*> *<;>* *<CompanyName[30]>* *<;>* *<ClientPINnum[14]>* *<;>* *<HeadQuarters[30]>* *<;>* *<Address[30]>* *<;>* *<PostalCodeAndCity[30]>* *<;>* *<ExemptionNum[30]>* *<;>*<TraderSystemInvNum[15]>

output: ACK

FPR operation: Opens a fiscal invoice receipt assigned to the specified operator number and operator password with free info for customer data. The Invoice receipt can be issued only if the invoice range (start and end numbers) is set.

Input data :

reserved	1 symbol with value '1'
reserved	1 symbol with value ' 0'
reserved	1 symbol with value '0'
InvoiceType	1 symbol with value '1'
CompanyName	30 symbols for Invoice company name
ClientPINnum	14 symbols for client PIN number
HeadQuarters	30 symbols for customer headquarters
Address	30 symbols for Address
PostalCodeAndCity	30 symbols for postal code and city
ExemptionNum	30 symbols for exemption number
TraderSystemInvNum	15 symbols for trader system invoice number

Output data: n. a.

Note:

If Postponed is enabled after opened fiscal receipt, all the next commands will be executed but won't be immediately. The data for whole receipt is stored to AS sent information for receipt closure. If up to 5 sec timeout no command ECR closing the receipt

zfpdef: *OpenInvoiceWithFreeCustomerData(CompanyName, ClientPINnum, HeadQuarters, Address, PostalCodeAndCity, ExemptionNum,TraderSystemInvNum)*

2.5.3. Command: 30h / 0 – Open Fiscal Invoice Credit Note receipt

input: <reserved*['1']*> <;> <*reserved[' 0']*> <;> <*reserved['0']*> <;> <*InvoiceDebitNoteType['A']*> <;> <*CompanyName[30]*> <;> <*ClientPINnum[14]*> <;> <*HeadQuarters[30]*> <;> <*Address[30]*> <;> <*PostalCodeAndCity[30]*> <;> <*ExemptionNum[30]*> <;> <*RelatedInvoiceNum[19]*><;><*TraderSystemInvNum[15]*>

output: ACK

FPR operation: Opens a fiscal invoice credit note receipt assigned to the specified operator number and operator password with free info for customer data. The Invoice receipt can be issued only if the invoice range (start and end numbers) is set.

Input data :

reserved	1 symbol with value '1'
reserved	1 symbol with value ' 0'
reserved	1 symbol with value '0'
InvoiceDebitNoteType	1 symbol with value 'A'
CompanyName	30 symbols for Invoice company name
ClientPINnum	14 symbols for client PIN number
HeadQuarters	30 symbols for customer headquarters
Address	30 symbols for Address
PostalCodeAndCity	30 symbols for postal code and city
ExemptionNum	30 symbols for exemption number
RelatedInvoiceNum	*19* symbols for the related invoice number in format ##########################
TraderSystemInvNum	15 symbols for trader system invoice number

Output data: n. a.

Note:

If Postponed is enabled after opened fiscal receipt, all the next commands will be executed but won't be immediately. The data for whole receipt is stored to AS sent information for receipt closure. If up to 5 sec timeout no command ECR closing the receipt

zfpdef: *OpenCreditNoteWithFreeCustomerData*(*CompanyName*, *ClientPINnum*, *HeadQuarters*, *Address, PostalCodeAndCity, ExemptionNum, RelatedInvoiceNum, TraderSystemInvNum*)

2.5.4. Command: 30h / 0 – Open Fiscal Invoice Debit Note receipt

input: <reserved*['1']*> *<;>* *<reserved[' 0']>* *<;>* *<reserved['0']>* *<;>* <InvoiceDebitNoteType['@']> *<;>* *<CompanyName[30]>* *<;>* *<ClientPINnum[14]>* *<;>* *<HeadQuarters[30]>* *<;>* *<Address[30]>* *<;>* *<PostalCodeAndCity[30]>* *<;>* *<ExemptionNum[30]>* *<;>* *<RelatedInvoiceNum[19]>*<;><*TraderSystemInvNum[15]*>

output: ACK

FPR operation: Opens a fiscal invoice debit note receipt assigned to the specified operator number and operator password with free info for customer data. The Invoice receipt can be issued only if the invoice range (start and end numbers) is set.

Input data :

reserved	1 symbol with value '1'
reserved	1 symbol with value ' 0'
reserved	1 symbol with value '0'
InvoiceDebitNoteType	1 symbol with value @
CompanyName	30 symbols for Invoice company name
ClientPINnum	14 symbols for client PIN number
HeadQuarters	30 symbols for customer headquarters
Address	30 symbols for Address
PostalCodeAndCity	30 symbols for postal code and city
ExemptionNum	30 symbols for exemption number
RelatedInvoiceNum	*19* symbols for the related invoice number in format ##########
TraderSystemInvNum	15 symbols for trader system invoice number

Output data: n. a.

Note:

If Postponing is enabled after opened fiscal receipt, all the next commands will be executed. The data for whole receipt is stored to AS sent information for receipt closure. If up to 5 sec timeout no command ECR closing the receipt

zfpdef: *OpenDebitNoteWithFreeCustomerData*(*CompanyName*, *ClientPINnum*, *HeadQuarters*, *Address, PostalCodeAndCity, ExemptionNum, RelatedInvoiceNum, TraderSystemInvNum)*

2.5.5. Command: 31h / 1 – Sell of article

input: *<NamePLU[36]> <;> <OptionVATClass[1]> <;> <Price[1..15]> <;> <MeasureUnit[3]> <;><HSCode[10]> <;> <HSName[20]> <;> <VATGrRate[1..5]> {<'*'> <Quantity[1..10]>} {<','> <DiscAddP[1..7]>}*

output: ACK

FPR operation: Register the sell (for correction use minus sign in the price field) of article with specified name, price, quantity, VAT class and/or discount/addition on the transaction.

Input data :

NamePLU	(PLU Name) 36 symbols for article's name
OptionVATClass	1 symbol for article's VAT class with optional values:" - 'A' – VAT Class A
Price	Up to 15 symbols for article's price with upto 5 digits after decimal point
MeasureUnit	3 symbols for measure unit
HSCode	(HS Code) 10 symbols for HS Code in format XXXX.XX.XX
HSName	(HS Name) 20 symbols for HS Name
VATGrRate	Up to 5 symbols for programmable VAT rate
'*'	1 symbol '*' indicating the presence of quantity field
Quantity	(Quantity)1 to 10 symbols for quantity
','	1 symbol ',' indicating the presence of discount/addition field
DiscAddP	(Discount/Addition %.) 1 to 7 for percentage of discount/addition

Output data : n. a.

zfpdef: SellPLUfromExtDB(NamePLU, OptionVATClass, Price, MeasureUnit, HSCode, HSName, Quantity, DiscAddP)

2.5.5. Command: 31h / 1 – Sell of article with defined HS code

input: *<NamePLU[36]> <;> <reserved[' ']> <;> <Price[1..15]> <;> <reserved[' ']> <;><HSCode[10]> <;> <reserved[' ']> <;> <reserved['0']> {<'*'> <Quantity[1..10]>} {<','> <DiscAddP[1..7]>}*

output: ACK

FPR operation: Register the sell (for correction use minus sign in the price field) of article with specified name, price, quantity, VAT class and/or discount/addition on the transaction.

Input data :

NamePLU	(PLU Name) 36 symbols for article's name
reserved	1 symbol reserved space symbol
Price	Up to 15 symbols for article's price with upto 5 digits after decimal point
reserved	3 symbols reserved space symbols
HSCode	(HS Code) 10 symbols for HS Code in format XXXX.XX.XX
reserved	20 symbols reserved space symbols
reserved	1 symbol '0'
'*'	1 symbol '*' indicating the presence of quantity field
Quantity	(Quantity)1 to 10 symbols for quantity
','	1 symbol ',' indicating the presence of discount/addition field
DiscAddP	(Discount/Addition %.) 1 to 7 for percentage of discount/addition

Output data : n. a.

zfpdef: SellPLUfromExtDB_HS()

2.5.6. Command: 2Ch / , – Open receipt, sell of article and close receipt

input: *<ReceiptType['0']><;><TraderSystemInvNum[15]><;><NamePLU[36]> <;>*

<OptionVATClass[1]> <;> <Price[1..15]> <;> <MeasureUnit[3]> <;><HSCode[10]>
{<''>* <Quantity*[1..10]>} {<','> <DiscAddP[1..7]>}*
output: <InvoiceNum[19]<;><QRcode[128]>

FPR operation: Opens receipt after which makes sell of article with specified name, price, quantity, VAT class and/or discount/addition on the transaction and then closes the receipt

Input data :

ReceiptType	1 symbols with fixed value of '0'
TraderSystemInvNum	15 symbols for the trader system invoice number
NamePLU	(PLU Name) 36 symbols for article's name
OptionVATClass	1 symbol for article's VAT class with optional values:" - 'A' – VAT Class A
Price	Up to 15 symbols for article's price with upto 5 digits after decimal point
MeasureUnit	3 symbols for measure unit
HSCode	(HS Code) 10 symbols for HS Code in format XXXX.XX.XX
ReservedHScode	1 symbol ' '
'*'	1 symbol '*' indicating the presence of quantity field
Quantity	(Quantity)1 to 10 symbols for quantity
','	1 symbol ',' indicating the presence of discount/addition field
DiscAddP	(Discount/Addition %.) 1 to 7 for percentage of discount/addition

Output data :

InvoiceNum	19 symbols for CU invoice number
QRcode	128 symbols for QR code

zfpdef: OpenSellAndCloseReceipt()

2.5.7. Command: 2Ch / , – Open Fiscal Invoice, sell of article and close receipt

input: <InvoiceType*['1']*> *<;> <ClientPINnum[14]> <;><ExemptionNum[15]> <;><TraderSystemInvNum[15]> <;><NamePLU[36]> <;> <OptionVATClass[1]> <;> <Price[1..15]> <;><MeasureUnit[3]> <;><HSCode[10]>*
{<''> <Quantity[1..10]>} {<','> <DiscAddP[1..7]>}*
output: <InvoiceNum[19]<;><QRcode[128]>
FPR operation: Opens invoice after which makes sell of article with specified name, price, quantity, VAT class and/or discount/addition on the transaction and then closes the invoice

Input data :

InvoiceType	1 symbols with fixed value of '1'
ClientPINnum	14 symbols for client PIN number
ExemptionNum	15 symbols for the exemption number
TraderSystemInvNum	15 symbols for the trader system invoice number
NamePLU	(PLU Name) 36 symbols for article's name
OptionVATClass	1 symbol for article's VAT class with optional values:" - 'A' – VAT Class A
Price	Up to 15 symbols for article's price with upto 5 digits after decimal point
MeasureUnit	3 symbols for measure unit
HSCode	(HS Code) 10 symbols for HS Code in format XXXX.XX.XX
ReservedHScode	1 symbol ' '
'*'	1 symbol '*' indicating the presence of quantity field
Quantity	(Quantity)1 to 10 symbols for quantity
','	1 symbol ',' indicating the presence of discount/addition field
DiscAddP	(Discount/Addition %.) 1 to 7 for percentage of discount/addition

Output data :

InvoiceNum	19 symbols for CU invoice number
QRcode	128 symbols for QR code

zfpdef: OpenSellAndCloseInvoice()

2.5.6. Command: 33h / 3 – Subtotal

input: *<OptionPrinting['0']> <;> <OptionDisplay[1]> {<':'> <DiscAddV[1..13]>} {<','> <DiscAddP[1..7]>}*

output:*<SubtotalValue[1..13]>*

FPR operation: Calculate the subtotal amount with printing and display visualization options. Provide information about values of the calculated amounts. If a percent or value discount/addition has been specified the subtotal and the discount/addition value will be printed regardless the parameter for printing.

Input data :

OptionPrinting	1 symbol with value *'0'*
OptionDisplay	1 symbol with value: - '1' – Yes - '0' – No
':'	1 symbol ':' indicating the presence of value discount/addition field
DiscAddV	(Discount/Addition Value) Up to 13 symbols for the value of the discount/addition. Use minus sign '-' for discount
','	1 symbol ',' indicating the presence of percent discount/addition field
DiscAddP	(Discount/Addition %) Up to 7 symbols for the percentage value of the discount/addition. Use minus sign '-' for discount

Output data:

SubtotalValue	Up to 13 symbols for the value of the subtotal amount

Notes:

When the discount/addition is a percentage the amount is distributed proportionally over the turnover items and is automatically transferred to the turnovers of the corresponding VAT groups.

A value discount/addition may be specified only if all sales are of articles (items) belonging to one and the same VAT group.

zfpdef: *Subtotal(OptionDisplay, DiscAddP, DiscAddV)*

2.5.7. Command: 38h / 8 – Close fiscal receipt

input: n. a.

output: <InvoiceNum[19]<;><QRcode[128]>

FPR operation: Closes the opened fiscal receipt and returns receipt info.

Input data : n. a.

Output data :

InvoiceNum	19 symbols for CU invoice number
QRcode	128 symbols for QR code

zfpdef: *CloseReceipt()*

2.5.8. Command: 39h / 9 – Cancel fiscal receipt

input: n. a.

output: ACK

FPR operation: Available only if the receipt is not closed. Cancel all sales in the receipt and close it .

Input data : n. a.

Output data : n. a.

zfpdef: *CancelReceipt()*

2.5.9. Command: 3Ah / : – Inform about issued receipt duplicate

input: n. a.

output: ACK

FPR operation: Informs about the issued document

Input data : n. a.

Output data : n. a.

zfpdef: InfoLastReceiptDuplicate()

2.6. COMMANDS FOR READING THE DATA IN CU'S REGISTERS

This set of commands provides information about the status of CU's registers without causing a device activity, i.e. the information is obtained through the communication interface without visualization.

2.6.1. Command: 6Dh / m – Read amounts by VAT Class

input: n. a.

output: *<SaleVATGrA[1..15]> <;> <SaleVATGrB[1..15]> <;> <SaleVATGrC[1..15]> <;><SaleVATGrD[1..15]> <;><SaleTurnoverVATGrE[1..15]> <;> <SaleTurnoverABCD[1..15]> <;> <RefundVATGrA[1..15]> <;> <RefundVATGrB[1..15]> <;> <RefundVATGrC[1..15]> <;> <RefundVATGrD[1..15]> <;> <RefundTurnoverVATGrE[1..15]> <;> <RefundTurnoverABCD[1..15]>*

FPR operation: Provides information about the accumulated amounts and refunded amounts by VAT class in case that CU regularly informs about the Z report(7C)

Input data : n. a.

Output data :

SaleVATGrA	Up to 15 symbols for the accumulated VAT in group A
SaleVATGrB	Up to 15 symbols for the accumulated VAT in group B
SaleVATGrC	Up to 15 symbols for the accumulated VAT in group C
SaleVATGrD	Up to 15 symbols for the accumulated VAT in group D
SaleTurnoverVATGrE	Up to 15 symbols for the accumulated turnover in group E
SaleTurnoverABCD	Up to 15 symbols for the sale turnover in VAT groups A, B, C, D
RefundVATGrA	Up to 15 symbols for the refund VAT in group A
RefundVATGrB	Up to 15 symbols for the refund VAT in group B
RefundVATGrC	Up to 15 symbols for the refund VAT in group C
RefundVATGrD	Up to 15 symbols for the refund VAT in group D
RefundTurnoverVATGrE	Up to 15 symbols for the refund accumulated turnover in group E
RefundTurnoverABCD	Up to 15 symbols for the refund turnover in VAT groups A, B, C, D

zfpdef: ReadDailyAmountsByVAT*()*

2.6.2. Command: 6Dh / m – Read EOD amounts, option 'd'

input: *<Option['d']>*

output: *<Option['d']> <;> <EOD_sale_turnover[1..20]> <;> <EOD_credit_turnover[1..20]> <;> <EOD_saleVAT[1..20]> <;> <EOD_creditVAT [1..20]> <;>*

FPR operation: Provides information about the accumulated EOD turnover and VAT

Input data :

Option	1 symbol with value 'd'

Output data :

Option	1 symbol with value 'd'
EOD_sale_turnover	Up to 20 symbols for the EOD sale turnover
EOD_credit_turnover	Up to 20 symbols for the EOD credit turnover
EOD_saleVAT	Up to 20 symbols for the EOD VAT from sales
EOD_creditVAT	Up to 20 symbols for the EOD VAT from credit invoices

zfpdef: ReadEODAmounts*()*

2.6.3. Command: 6Dh / m – Read first EOD date

input: *<Option['d']> <;> <Option['f']>*

output: *<Option['d']> <;> <Option['f']> <;> <DateTime "DDMMYY">*

FPR operation: Provides information about first EOD date

Input data :

Option	1 symbol with value 'd'
Option	1 symbol with value 'f'

Output data :

Option	1 symbol with value 'd'
DateTime	Date Time parameter in format: *DDMMYY*

zfpdef: ReadFirstEODDate*()*

2.6.4. Command: 6Dh / m – Read EOD amounts by date

input: *<Option['d']> <;> <EOD_Date "*DDMMYY*">*

output: *<Option['d']> <;> <EOD_Date "*DDMMYY*"> <;> <EOD_sale_turnover[1..20]> <;> <EOD_credit_turnover[1..20]> <;> <EOD_saleVAT[1..20]> <;> <EOD_creditVAT [1..20]> <;> <InvoiceCount[4]> <;> <CreditCount[4]>*

FPR operation: Provides information about the accumulated EOD turnover and VAT amounts by date

Input data :

Option	1 symbol with value 'd'
EOD_Date	6 symbols for initial date in the DDMMYY format

Output data :

Option	1 symbol with value 'd'
EOD_Date	6 symbols for initial date in the DDMMYY format
EOD_sale_turnover	Up to 20 symbols for the EOD sale turnover
EOD_credit_turnover	Up to 20 symbols for the EOD credit turnover
EOD_saleVAT	Up to 20 symbols for the EOD VAT from sales
EOD_creditVAT	Up to 20 symbols for the EOD VAT from credit invoices
InvoiceCount	4 symbols for invoices count
CreditCount	4 symbols for credit count

zfpdef: ReadEODAmountsByDate*()*

2.6.5. Command: 71h / q – Read receipt numbers

input: n. a.

output: *<LastCUInvoiceNum[19]> <;> <LastReceiptNum[7]>*

FPR operation: Provides information about the number of the last issued receipt.

Input data : n. a.

Output data :

LastCUInvoiceNum	19 symbols for the last number of invoice according the middleware, CU, internal invoice counter
LastReceiptNum	7 symbols for last receipt number in format ########

zfpdef: *ReadLastAndTotalReceiptNum()*

2.6.6. Command: 72h / r – Read information about the current opened receipt

input: n. a.

output:<*IsReceiptOpened[1]*> <;> <*SalesNumber[3]*> <;> <*SubtotalAmountVATGA[1..13]*> <;> <*SubtotalAmountVATGB[1..13]*> <;> <*SubtotalAmountVATGC[1..13]*> <;> <*SubtotalAmountVATGD[1..13]*> <;> <*SubtotalAmountVATGE[1..13]*> <;> <*reserved['0']*> <;><*reserved['0']*> <;> <*ReceiptFormat[1]*> <;> <*reserved['0']*> <;> <*reserved['0']*> <;> <*ClientReceipt[1]*> <;> <*PowerDownInReceipt[1]*> <;> <*reserved['0']*>

FPR operation: Read the current status of the receipt.

Input data : n. a.

Output data :

IsReceiptOpened	*1 symbol with value:* *- '0' - No* *- '1' - Yes*
SalesNumber	3 symbols for number of sales
SubtotalAmountVATGA	Up to 13 symbols for subtotal by VAT group A
SubtotalAmountVATGB	Up to 13 symbols for subtotal by VAT group B
SubtotalAmountVATGC	Up to 13 symbols for subtotal by VAT group C
SubtotalAmountVATGD	Up to 13 symbols for subtotal by VAT group D
SubtotalAmountVATGE	Up to 13 symbols for subtotal by VAT group E
reserved	1 symbol with value '0'
reserved	1 symbol with value '0'
ReceiptFormat	(Format) 1 symbol with value: - '1' - Detailed - '0' - Brief
reserved	1 symmbol with value '0'
reserved	1 symbol with value '0'
ClientReceipt	1 symbol with value: - '1' – invoice (client) receipt - '0' – standard receipt
PowerDownInReceipt	1 symbol with value: - '0' - No - '1' - Yes
reserved	1 symbol with value '0'

zfpdef: ReadCurrentReceiptInfo()

2.7. Reports commands

Set of commands for reports generated by CU.

2.7.1. Command: 7Ch / | – Inform for daily fiscal Z report

Input: *<'Z'>*
Output: *ACK*
FPR Operation: Provides information for the daily fiscal report with zeroing and fiscal memory record, preceded by Electronic Journal report.

Input data:

'Z'	1 symbol with value Z

Output data: n.a.
zfpdef: *DailyReport()*

2.7.2. Command: 7Ch / | –Store Electronic Journal report from date to date

input:*<OptionReportStorage[2]> <;> <'D'> <;> <StartRepFromDate "*DDMMYY*"> <;> <EndRepFromDate "*DDMMYY*">*
output: ACK
FPR operation: Store Electronic Journal Report from report from date to date to External USB, External SD card.

Input data:

OptionReportStorage	(Store EJ Report) 2 symbols for destination: - 'J4' – Storage in External SD card memory - 'JX' – Storage in External SD card memory for JSON
'D'	1 symbol 'D'
StartRepFromDate	(Start date) 6 symbols for initial date in the DDMMYY format
EndRepFromDate	(End date) 6 symbols for final date in the DDMMYY format

Output data: n. a.
zfpdef: StoreEJByDate(*OptionReportStorage*, StartDate, EndDate)

2.7.3. Command: 7Ch / | –Store Electronic Journal report from datetime to datetime

input:*<OptionReportStorage[2]> <;> <ReportFormat[1]> <;> <StartRepFromDate "*DD-MM-YYYY HH:MM*"> <;> <EndRepFromDate "*DD-MM-YYYY HH:MM*">*
output: ACK
FPR operation: Store Electronic Journal Report from report from datetime to datetime to External SD card.

Input data:

OptionReportStorage	(Store EJ Report) 2 symbols for destination: - 'J4' – Storage in External SD card memory - 'JX' – Storage in External SD card memory for JSON
ReportFormat	(EJ Report format) 1 character with value - 'T' – Detailed EJ - 't' – Brief EJ
StartRepFromDate	(Start datetime) 16 symbols for initial date in the DD-MM-YYYY HH:MM format
EndRepFromDate	(End datetime) 16 symbols for final date in the DD-MM-YYYY HH:MM format

Output data: n. a.
zfpdef: StoreEJByDateTime(*OptionReportStorage*, ReportFormat, StartDateTime, EndDateTime)

2.7.4. Command: 7Ch / | –Store Electronic Journal report by CU Invoice Number

input:<*OptionReportStorage[2]*> <;> <*InvoiceReportFormat[1]*> <;> <*StartCUInvoiceNum[10]*> <;> <*EndCUInvoiceNum[10]*>

output: ACK

FPR operation: Store Electronic Journal Report by CU Invoice Number (Multiple invoices) to External SD card.

Input data:

OptionReportStorage	(Store EJ Report) 2 symbols for destination: - 'J4' – Storage in External SD card memory - 'JX' – Storage in External SD card memory for JSON
InvoiceReportFormat	(EJ Report format) 1 character with value - 'N' – Detailed EJ - 'n' – Brief EJ
StartCUInvoiceNum	10 symbols for start CU invoice number in format: ##########.
EndCUInvoiceNum	10 symbols for end CU invoice number in format: ##########.

Output data: n. a.

zfpdef: StoreEJByCUInvoiceNum(*OptionReportStorage*, InvoiceReportFormat, StartCUInvoiceNum, EndCUInvoiceNum)

2.7.5. Command: 7Ch / | –Store Electronic Journal report by PIN of buyer

input:<*OptionReportStorage[2]*> <;> <*PINReportFormat[1]*> <;> <*PINnumber[11]*>

output: ACK

FPR operation: Store Electronic Journal Report by PIN number of buyer to External SD card.

Input data:

OptionReportStorage	(Store EJ Report) 2 symbols for destination: - 'J4' – Storage in External SD card memory - 'JX' – Storage in External SD card memory for JSON
PINReportFormat	(EJ Report format) 1 character with value - 'P' – Detailed EJ - 'p' – Brief EJ
PINnumber	(PIN of buyer) 11 symbols for PIN number

Output data: n. a.

zfpdef: StoreEJByPINofBuyer (*OptionReportStorage*, PINReportFormat, PINnumber)

2.7.6. Command: 7Ch / | –Store Electronic Journal report by Trader System Invoice Number

input:<*OptionReportStorage[2]*> <;> <*TraderSystemReportFormat[1]*> <;> <*TraderSystemInvNum[15]*>

output: ACK

FPR operation: Store Electronic Journal Report by Trader System Invoice Number (Single Invoice) to External SD card.

Input data:

OptionReportStorage	(Store EJ Report) 2 symbols for destination: - 'J4' – Storage in External SD card memory - 'JX' – Storage in External SD card memory for JSON
TraderSystemReportFormat	(EJ Report format) 1 character with value - 'S' – Detailed EJ - 's' – Brief EJ
TraderSystemInvNum	(Invoice Number) 15 symbols for trader system invoice number

Output data: n. a.

zfpdef: StoreEJByTraderSystemInvNum(*OptionReportStorage*, TraderSystemReportFormat, TraderSystemInvNum)

2.7.7. Command: 7Ch / | –Store Electronic Journal report from beginning to end

input: *<OptionReportStorage[2]><;><'*'>*
output: ACK
FPR operation: Store whole Electronic Journal report to External SD card.

Input data:

OptionReportStorage	(Store EJ Report) 2 symbols for destination: - 'J4' – Storage in External SD card memory - 'JX' – Storage in External SD card memory for JSON
''*	1 symbol '*'

Output data: n. a.
zfpdef: StoreEJ()

2.7.8. Command: 7Ch / | – Read Electronic Journal report from date to date

input: *<'J0'> <;> <'D'> <;> <StartRepFromDate "*DDMMYY*"><;> <EndRepFromDate "*DDMMYY*">*
output: ACK+
FPR operation: Read Electronic Journal Report by initial to end date.

Input data:

'J0'	(EJ Report storage) 2 characters with value *'J0'*
'D'	1 symbol 'D'
StartRepFromDate	(Start date) 6 symbols for initial date in the DDMMYY format
EndRepFromDate	(End date) 6 symbols for final date in the DDMMYY format

Output data: n. a.
zfpdef: ReadEJByDate(StartDate, EndDate)

2.7.9. Command: 7Ch / | – Read Electronic Journal report from date to date in JSON format

input: *<'JY'> <;> <'D'> <;> <StartRepFromDate "*DDMMYY*"><;> <EndRepFromDate "*DDMMYY*">*
output: *<'JY'> <;> <'Z'> <;> <Length[1..10]>*
FPR operation: Read Electronic Journal Report by initial to end date in the JSON format. The number of bytes equal to *Length* for JSON data must be read subsequently.

Input data:

'JY'	(EJ Report storage) 2 characters with value *'JY'*
'D'	1 symbol with value 'D'
StartRepFromDate	(Start date) 6 symbols for initial date in the DDMMYY format
EndRepFromDate	(End date) 6 symbols for final date in the DDMMYY format

Output data:

'JY'	(EJ Report storage) 2 characters with value *'JY'*
'Z'	1 symbol with value 'Z'
Length	Up to 10 symbols for length

zfpdef: ReadEJByDate_JSON(StartDate, EndDate)

2.7.10. Command: 7Ch / | – Read Electronic Journal report from datetime to datetime

input: *<'J0'> <;> <ReportFormat[1]> <;> <StartRepFromDate "*DD-MM-YYYY HH:MM*"><;> <EndRepFromDate "*DD-MM-YYYY HH:MM*">*

output: ACK+

FPR operation: Read Electronic Journal Report by initial to end date and time.

Input data:

'J0'	(EJ Report storage) 2 characters with value *'J0'*
ReportFormat	(EJ Report format) 1 character with value - 'T' – Detailed EJ - 't' – Brief EJ
StartRepFromDate	(Start date and time) 16 symbols for initial date in the DD-MM-YYYY HH:MM format
EndRepFromDate	(End date and time) 16 symbols for final date in the DD-MM-YYYY HH:MM format

Output data: n. a.

zfpdef: ReadEJByDateTime(ReportFormat, StartDateTime, EndDateTime)

2.7.11. Command: 7Ch / | – Read Electronic Journal report from datetime to datetime in JSON format

input: *<'JY'> <;> <ReportFormat[1]> <;> <StartRepFromDate "*DD-MM-YYYY HH:MM*"><;> <EndRepFromDate "*DD-MM-YYYY HH:MM*">*

output: *<'JY'> <;> <'Z'> <;> <Length[1..10]>*

FPR operation: Read Electronic Journal Report by initial to end date and time in the JSON format. The number of bytes equal to *Length* for JSON data must be read subsequently.

Input data:

'JY'	(EJ Report storage) 2 characters with value *'JY'*
ReportFormat	(EJ Report format) 1 character with value - 'T' – Detailed EJ - 't' – Brief EJ
StartRepFromDate	(Start date and time) 16 symbols for initial date in the DD-MM-YYYY HH:MM format
EndRepFromDate	(End date and time) 16 symbols for final date in the DD-MM-YYYY HH:MM format

Output data:

'JY'	(EJ Report storage) 2 characters with value *'JY'*
'Z'	1 symbol with value 'Z'
Length	Up to 10 symbols for length

zfpdef: ReadEJByDateTime_JSON(ReportFormat, StartDateTime, EndDateTime)

2.7.12. Command: 7Ch / | – Read Electronic Journal report by CU Invoice Number

input: *<'J0'> <;> <InvoiceReportFormat[1]> <;> <StartCUInvoiceNum[10]> <;> <EndCUInvoiceNum[10]>*

output: ACK+

FPR operation: Read Electronic Journal Report by CU Invoice Number (Multiple invoices)

Input data:

'J0'	(EJ Report storage) 2 characters with value *'J0'*
InvoiceReportFormat	(EJ Report format) 1 character with value - 'N' – Detailed EJ - 'n' – Brief EJ
StartCUInvoiceNum	10 symbols for start CU invoice number in format: ##########.
EndCUInvoiceNum	10 symbols for end CU invoice number in format: ##########.

Output data: n. a.

zfpdef: ReadEJByCUInvoiceNum(InvoiceReportFormat, StartCUInvoiceNum, EndCUInvoiceNum)

2.7.13. Command: 7Ch / | – Read Electronic Journal report by CU Invoice Number in JSON format

input: *<'JY'> <;> < InvoiceReportFormat[1]> <;> <StartCUInvoiceNum[10]> <;> <EndCUInvoiceNum[10]>*

output: *<'JY'> <;> <'Z'> <;> <Length[1..10]>*

FPR operation: Read Electronic Journal Report by CU Invoice Number (Multiple invoices) in the JSON format. The number of bytes equal to *Length* for JSON data must be read subsequently.

Input data:

'J0'	(EJ Report storage) 2 characters with value *'JY'*
InvoiceReportFormat	(EJ Report format) 1 character with value - 'N' – Detailed EJ - 'n' – Brief EJ
StartCUInvoiceNum	10 symbols for start CU invoice number in format: ##########.
EndCUInvoiceNum	10 symbols for end CU invoice number in format: ##########.

Output data:

'JY'	(EJ Report storage) 2 characters with value *'JY'*
'Z'	1 symbol with value 'Z'
Length	Up to 10 symbols for length

zfpdef: ReadEJByCUInvoiceNum_JSON(InvoiceReportFormat, StartCUInvoiceNum, EndCUInvoiceNum)

2.7.14. Command: 7Ch / | – Read Electronic Journal report by PIN of buyer

input: *<'J0'> <;> <PINReportFormat[1]> <;> <PINnumber[11]>*

output: ACK+

FPR operation: Read Electronic Journal Report by PIN number of buyer.

Input data:

'J0'	(EJ Report storage) 2 characters with value *'J0'*
PINReportFormat	(EJ Report format) 1 character with value - 'P' – Detailed EJ - 'p' – Brief EJ
PINnumber	(PIN of buyer) 11 symbols for PIN number

Output data: n. a.

zfpdef: ReadEJByPINofBuyer(PINReportFormat, PINnumber)

2.7.15. Command: 7Ch / | – Read Electronic Journal report by PIN of buyer in JSON format

input: *<'JY'> <;> < PINReportFormat[1]> <;> <PINnumber[11]>*

output: *<'JY'> <;> <'Z'> <;> <Length[1..10]>*

FPR operation: Read Electronic Journal Report by PIN number of buyer in the JSON format. The number of bytes equal to *Length* for JSON data must be read subsequently.

Input data:

'JY'	(EJ Report storage) 2 characters with value *'JY'*
PINReportFormat	(EJ Report format) 1 character with value - 'P' – Detailed EJ - 'p' – Brief EJ
PINnumber	(PIN of buyer) 11 symbols for PIN number

Output data:

'JY'	(EJ Report storage) 2 characters with value *'JY'*
'Z'	1 symbol with value 'Z'
Length	Up to 10 symbols for length

zfpdef: ReadEJByPINofBuyer_JSON(PINReportFormat, PINnumber)

2.7.16. Command: 7Ch / | – Read Electronic Journal report by Trader System Invoice Number

input: *<'J0'> <;> <TraderSystemReportFormat[1]> <;> <TraderSystemInvNum[15]>*
output: ACK+
FPR operation: Read Electronic Journal Report by Trader System Invoice Number (Single Invoice).

Input data:

'J0'	(EJ Report storage) 2 characters with value *'J0'*
TraderSystemReportFormat	(EJ Report format) 1 character with value - 'S' – Detailed EJ - 's' – Brief EJ
TraderSystemInvNum	(Invoice Number) 15 symbols for trader system invoice number

Output data: n. a.

zfpdef: ReadEJByTraderSystemInvNum(TraderSystemReportFormat, TraderSystemInvNum)

2.7.17. Command: 7Ch / | – Read Electronic Journal report by Trader System Invoice Number in JSON format

input: *<'JY'> <;> < TraderSystemReportFormat[1]> <;> <TraderSystemInvNum[15]>*
output: *<'JY'> <;> <'Z'> <;> <Length[1..10]>*
FPR operation: Read Electronic Journal Report by Trader System Invoice Number (Single Invoice) in the JSON format. The number of bytes equal to *Length* for JSON data must be read subsequently.

Input data:

'JY'	(EJ Report storage) 2 characters with value *'JY'*
TraderSystemReportFormat	(EJ Report format) 1 character with value - 'S' – Detailed EJ - 's' – Brief EJ
TraderSystemInvNum	(Invoice Number) 15 symbols for trader system invoice number

Output data:

'JY'	(EJ Report storage) 2 characters with value *'JY'*
'Z'	1 symbol with value 'Z'
Length	Up to 10 symbols for length

zfpdef: ReadEJByTraderSystemInvNum_JSON(TraderSystemReportFormat, TraderSystemInvNum)

2.7.18. Command: 7Ch / | – Read Electronic Journal report from beginning to end

input: *<'J0'> <;> <'*'>*
output: ACK+
FPR operation: Read whole Electronic Journal report from beginning to the end.

Input data:

'J0'	(EJ Report storage) 2 characters with value *'J0'*
''*	1 symbol '*'

Output data: n. a.

zfpdef: ReadEJ()

2.7.19. Command: 7Ch / | – Read Electronic Journal report from beginning to end in JSON format

input: *<'JY'> <;> <'*'>*
output: *<'JY'> <;> <'*'> <;> <Length[1..10]>*
FPR operation: Read whole Electronic Journal report from beginning to the end in the JSON format. The number of bytes equal to *Length* for JSON data must be read subsequently.

Input data:

'JY'	(EJ Report storage) 2 characters with value *'JY'*
''*	1 symbol '*'

Output data:

'JY'	(EJ Report storage) 2 characters with value *'JY'*
''*	1 symbol with value '*'
Length	Up to 10 symbols for length

zfpdef: ReadEJ_JSON()

2.7.20. Command: 7Ch / | – Read/Store Copy of Invoice receipt

input: *<OptionInvoiceCopy[2]><;><'I'><;> <CUInvoiceNum[10]>*

output: ACK+
FPR operation: Read/Store Invoice receipt copy to External SD card.

Input data:

OptionInvoiceCopy	(Printing/Storage Invoice receipt copy) 2 symbols for destination: - 'J0' – Reading - 'J4' – Storage in External SD card memory
'I'	1 symbol 'I'
CUInvoiceNum	10 symbols for Invoice receipt Number.

Output data: n. a.

zfpdef: ReadOrStoreInvoiceCopy(CUInvoiceNum)

2.8. Settings LAN/WiFi/GPRS commands

Set/Get commands for programming/reading of CU settings for LAN, WiFi, Bluetooth, GRPS.

2.8.1. Command: 4Eh / N – Read Device modules support by Firmware

input: *<'R'><;><'D'><;><'S'>*

output: *<'R'><;><'D'><;><'S'><;><LAN[1]><;><WiFi>[1]><;><GPRS>[1]><;><BT[1]>*

FPR operation: FlagsModule is a char with bits representing modules supported by the firmware

Input data:

'R'	1 symbol with value 'R'
'D'	1 symbol with value 'D'
'S'	1 symbol with value 'S'

Output data:

'R'	1 symbol with value 'R'
'D'	1 symbol with value 'D' for installed modules
'S'	1 symbol with value 'S' for firmware device support
LAN	1 symbol for LAN suppor - '0' – No - '1' – Yes
WiFi	1 symbol for WiFi support - '0' – No - '1' – Yes
GPRS	1 symbol for GPRS support - '0' – No - '1' – Yes
BT	(Bluetooth) 1 symbol for Bluetooth support - '0' – No - '1' – Yes

zfpdef: ReadDeviceModuleSupportByFirmware()

2.8.2. Command: 4Eh / N – Read Device modules support

input: *<'R'><;><'D'><;><'D'>*
output: *<'R'><;><'D'><;><'D'><;><LAN[1]><;><WiFi>[1]><;><GPRS>[1]><;><BT[1]>*
FPR operation: FlagsModule is a char with bits representing modules supported by the device.

Input data:

'R'	1 symbol with value 'R'
'D'	1 symbol with value 'D'
'D'	1 symbol with value 'D'

Output data:

'R'	1 symbol with value 'R'
'D'	1 symbol with value 'D' for installed modules
'D'	1 symbol with value 'D' for firmware device support
LAN	1 symbol for LAN suppor - '0' – No - '1' – Yes
WiFi	1 symbol for WiFi support - '0' – No - '1' – Yes
GPRS	1 symbol for GPRS support - '0' – No - '1' – Yes
BT	(Bluetooth) 1 symbol for Bluetooth support - '0' – No - '1' – Yes

zfpdef: ReadDeviceModuleSupport()

2.8.3. Command: 4Eh / N – Read TCP password

input: *<'R'><;><'Z'><;><'1'>*
output: *<'R'><;><'Z'><;><'1'><;><PassLength[1..3]><;><Password[100]>*
FPR operation: Provides information about device's TCP password.

Input data:

'R'	1 symbol with value 'R'
'Z'	1 symbol with value 'Z'
'1'	1 symbol with value '1'

Output data:

'R'	1 symbol with value 'R'
'Z'	1 symbol with value 'Z'
'1'	1 symbol with value '1'
PassLength	Up to 3 symbols for the password length
Password	(Password) Up to 100 symbols for the TCP password

zfpdef: ReadTCP_Password()

2.8.4. Command: 4Eh / N – Read TCP auto start

input: *<'R'><;><'Z'><;><'2'>*
output: *<'R'><;><'Z'><;><'2'><;><TCPAutoStart[1]>*
FPR operation: Provides information about if the TCP connection autostart when the device enter in Line/Sale mode.

Input data:

'R'	1 symbol with value 'R'
'Z'	1 symbol with value 'Z'
'2'	1 symbol with value '2'

Output data:

'R'	1 symbol with value 'R'
'Z'	1 symbol with value 'Z'
'2'	1 symbol with value '1'
TCPAutoStart	1 symbol for TCP auto start option - '0' – No - '1' – Yes

zfpdef: ReadTCP_AutoStartStatus()

2.8.5. Command: 4Eh / N – Read Device TCP addresses

input: *<'R'><;><'T'><;><AddressType[1]>*
output: *<'R'><;><'T'><;>< AddressType[1]><;><DeviceAddress[15]>*
FPR operation: Provides information about device's network IP address, subnet mask, gateway address, DNS address.

Input data:

'R'	1 symbol with value 'R'
'T'	1 symbol with value *'T'*
AddressType	(Address type) 1 symbol with value: - '2' – IP address - '3' – Subnet Mask - '4' – Gateway address - '5' – DNS address

Output data:

'R'	1 symbol with value 'R'
'T'	1 symbol with value *'T'*
AddressType	(Address type) 1 symbol with value: - '2' – IP address - '3' – Subnet Mask - '4' – Gateway address - '5' – DNS address
DeviceAddress	15 symbols for the device's addresses

zfpdef: ReadDeviceTCP_Addresses(AddressType)

2.8.6. Command: 4Eh / N – Read Device TCP MAC address

input: *<'R'><;><'T'><;><'6' >*
output: *<'R'><;><'T'><;><'6'><;><MACAddress[12]>*
FPR operation: Provides information about device's MAC address.

Input data:

'R'	1 symbol with value 'R'
'T'	1 symbol with value *'T'*
'6'	1 symbol with value '6'

Output data:

'R'	1 symbol with value 'R'
'T'	1 symbol with value *'T'*
'6'	1 symbol with value '6'
MACAddress	12 symbols for the device's MAC address

zfpdef: ReadTCP_MACAddress()

2.8.7. Command: 4Eh / N – Read Device NTP address

input: *<'R'><;><'S'><;><'N' >*
output: *<'R'><;><'S'><;><'N'><;><AddressLen[1..3]><;><NTPAddress[50]>*
FPR operation: Provides information about device's NTP address.

Input data:

'R'	1 symbol with value 'R'
'S'	1 symbol with value *'S'*
'N'	1 symbol with value 'N'

Output data:

'R'	1 symbol with value 'R'
'S'	1 symbol with value *'S'*
'N'	1 symbol with value 'N'
AddressLen	Up to 3 symbols for the address length
NTPAddress	(NTP Address)50 symbols for the device's NTP address

zfpdef: ReadNTP_Address ()

2.8.8. Command: 4Eh / N – Read TCP DHCP enabled

input: *<'R'><;><'T'><;><'1'>*
output: *<'R'><;><'T'><;><'1'><;><DHCPEnabled[1]>*
FPR operation: Provides information about device's DHCP status

Input data:

'R'	1 symbol with value 'R'
'T'	1 symbol with value *'T'*
'1'	1 symbol with value '*1*'

Output data:

'R'	1 symbol with value 'R'
'T'	1 symbol with value *'T'*
'1'	1 symbol with value '*1*'
DHCPEnabled	(Status) 1 symbols for device's DHCP status - '0' – Disabled - '1' – Enabled

zfpdef: ReadDHCP_Status()

2.8.9. Command: 4Eh / N – Scan for available WiFi networks

input: *<'R'><;><'W'><;><'S'>*
output: *ACK*
FPR operation: The device scan out the list of available WiFi networks.

Input data:

'R'	1 symbol with value 'R'
'W'	3 symbols with value *'GPR'*
'S'	7 symbols with value '*wf-scan*'

Output data:n. a.
zfpdef: ScanWiFiNetworks()

2.8.10. Command: 4Eh / N – Read TCP WiFi network name

input: *<'R'><;><'W'><;><'N'>*
output: *<'R'><;><'W'><;><'N'><;><WiFiNameLength[1..3]><;><WiFiNetworkName[100]>*
FPR operation: Provides information about WiFi network name where the device is connected.

Input data:

'R'	1 symbol with value 'R'
'W'	1 symbol with value *'W'*
'N'	1 symbol with value 'N'

Output data:

'R'	1 symbols with value *'R'*
'W'	1 symbols with value 'W'
'N'	1 symbol with value 'N'
WiFiNameLength	Up to 3 symbols for the WiFi name length
WiFiNetworkName	(Name) Up to 100 symbols for the device's WiFi network name

zfpdef: ReadWiFi_NetworkName()

2.8.11. Command: 4Eh / N – Read TCP WiFi password

input: *<'R'><;><'W'><;><'P'>*
output: *<'R'><;><'W'><;><'P'><;><PassLength[1..3]><;><Password[100]>*
FPR operation: Providing information about WiFi password where the device is connected.

Input data:

'R'	1 symbol with value 'R'
'W'	1 symbol with value *'W'*
'P'	1 symbol with value 'P'

Output data:

'R'	1 symbol with value 'R'
'W'	1 symbol with value *'W'*
'P'	1 symbol with value 'P'
PassLength	Up to 3 symbols for the WiFi password length
Password	Up to 100 symbols for the device's WiFi password

zfpdef: ReadWiFi_Password()

2.8.12. Command: 4Eh / N – Read TCP used communication module, LAN or WiFi

input: *<'R'><;><'Z'><;><'U'>*
output: *<'R'><;><'Z'><;><'U'><;><UsedModule[1]>*
FPR operation: Provides information about which module the device is in use: LAN or WiFi module. This information can be provided if the device has mounted both modules.

Input data:

'R'	1 symbol with value 'R'
'Z'	1 symbol with value *'Z'*
'U'	1 symbol with value 'U'

Output data:

'R'	1 symbol with value 'R'
'Z'	1 symbol with value *'Z'*
'U'	1 symbol with value 'U'
UsedModule	1 symbol with value: - '1' – LAN module - '2' – WiFi module

zfpdef: ReadTCP_UsedModule()

2.8.13. Command: 4Eh / N – Read TCP idle timeout

input: *<'R'><;><'Z'><;><'I'>*
output: *<'R'><;><'Z'><;><'I'><;><IdleTimeout[4]>*
FPR operation: Provides information about device's idle timeout. This timeout is seconds in which the connection will be closed when there is an inactivity. This information is available if the device has LAN or WiFi. Maximal value – 7200, minimal value 1. 0 is for never close the connection.

Input data:

'R'	1 symbol with value 'R'
'B'	1 symbol with value *'B'*
'I'	1 symbol with value 'I'

Output data:

'R'	1 symbol with value 'R'
'B'	1 symbol with value *'B'*
'I'	1 symbol with value 'I'
IdleTimeout	4 symbols for password in format ####

zfpdef: Read_IdleTimeout()

2.8.14. Command: 4Eh / N – Read GPRS APN

input: *<'R'><;><'G'><;><'A'>*
output: *<'R'><;><'G'><;><'A'><;><gprsAPNlength[1..3]><;><APN[100]>*
FPR operation: Provides information about device's GRPS APN.

Input data:

'R'	1 symbol with value 'R'
'G'	1 symbol with value *'G'*
'A'	1 symbol with value 'A'

Output data:

'R'	1 symbol with value 'R'
'G'	1 symbol with value *'G'*
'A'	1 symbol with value 'A'
gprsAPNlength	Up to 3 symbols for the APN length
APN	(APN) Up to 100 symbols for the device's GPRS APN

zfpdef: ReadGPRS_APN()

2.8.15. Command: 4Eh / N – Read GPRS password

input: *<'R'><;><'G'><;><'P'>*
output: *<'R'><;><'G'><;><'P'><;><PassLength[1..3]><;><Password[100]>*
FPR operation: Provides information about device's GPRS password.

Input data:

'R'	1 symbol with value 'R'
'G'	1 symbol with value *'G'*
'P'	1 symbol with value 'P'

Output data:

'R'	1 symbol with value 'R'
'G'	1 symbol with value *'G'*
'P'	1 symbol with value 'P'
PassLength	Up to 3 symbols for the GPRS password length
Password	Up to 100 symbols for the device's GPRS password

zfpdef: ReadGPRS_Password()

2.8.16. Command: 4Eh / N – Read GPRS username

input: *<'R'><;><'G'><;><'U'>*
output: *<'R'><;><'G'><;><'U'><;><gprsUserNameLength[1..3]><;><Username[100]>*
FPR operation: Providing information about device's GPRS user name.

Input data:

'R'	1 symbol with value 'R'
'G'	1 symbol with value *'G'*
'U'	1 symbol with value 'U'

Output data:

'R'	1 symbol with value 'R'
'G'	1 symbol with value *'G'*
'U'	1 symbol with value 'U'
gprsUserNameLength	Up to 3 symbols for the GPRS username length
Username	Up to 100 symbols for the device's GPRS username

zfpdef: ReadGPRS_Username()

2.8.17. Command: 4Eh / N – Read HTTPS server address

input: *<'R'><;><'S'><;><'H'>*
output: *<'R'><;><'S'><;><'C'><;><ParamLength[1..3]><;><Address[50]>*
FPR operation: Providing information about server HTTPS address.

Input data:

'R'	1 symbol with value 'R'
'S'	1 symbol with value *'S'*
'H'	1 symbol with value 'H'

Output data:

'R'	1 symbol with value 'R'
'S'	1 symbol with value *'S'*
'C'	1 symbol with value 'C'
ParamLength	Up to 3 symbols for parameter length
Address	50 symbols for address

zfpdef: ReadHTTPS_Server()

2.8.18. Command: 4Eh / N – Read Invoice threshold count

input: *<'R'><;><'S'><;><'I'>*
output: *<'R'><;><'S'><;><'I'><;><Value[1..5]>*
FPR operation: Read invoice threshold count

Input data:

'R'	1 symbol with value 'R'
'S'	1 symbol with value *'S'*
'I'	1 symbol with value 'I'

Output data:

'R'	1 symbol with value 'R'
'S'	1 symbol with value *'S'*
'I'	1 symbol with value 'I'
Value	Up to 5 symbols for value

zfpdef: ReadInvoice_Threshold()

2.8.19. Command: 4Eh / N – Read time threshold minutes

input: *<'R'><;><'S'><;><'T'>*
output: *<'R'><;><'S'><;><'T'><;><Value[1..5]>*
FPR operation: Read time threshold minutes

Input data:

'R'	1 symbol with value 'R'
'S'	1 symbol with value *'S'*
'T'	1 symbol with value 'T'

Output data:

'R'	1 symbol with value 'R'
'S'	1 symbol with value *'S'*
'T'	1 symbol with value 'T'
Value	Up to 5 symbols for value

zfpdef: ReadTimeThreshold_Minutes()

2.8.20. Command: 4Eh / N – Read Server Communication Module

input: *<'R'><;><'S'><;><'E'>*
output: *<'R'><;><'S'><;><'E'><;><Module [1]>*
FPR operation: Read device communication usage with server

Input data:

'R'	1 symbol with value 'R'
'S'	1 symbol with value *'Z'*
'E'	1 symbol with value '*E*'

Output data:

'R'	1 symbol with value 'R'
'S'	1 symbol with value *'Z'*
'E'	1 symbol with value '*E*'
Module	1 symbol with value: - '0' - GSM - '1' – LAN/WiFi

zfpdef: ReadServer_UsedComModule()

2.8.21. Command: 4Eh / N – Read total message count

input: *<'R'><;><'L'><;><'0'>*
output: *<'R'><;><'L'><;><'0'><;><Count[3]>*
FPR operation: Reads all messages from log

Input data:

'R'	1 symbol with value 'R'
'L'	1 symbol with value *'L'*
'0'	1 symbol with value '*0*'

Output data:

'R'	1 symbol with value 'R'
'L'	1 symbol with value *'L'*
'0'	1 symbol with value '*0*'
Count	3 symbols for the messages count

zfpdef: ReadTotalMessagesCount()

2.8.22. Command: 4Eh / N – Read specific message

input: *<'R'><;><'L'><;><MessageNum[2]>*
output: *<'R'><;><'L'><;><MessageNum[2]><;>*
<DateTime "DD-MM-YYYY HH:MM"> <;><Type[1]><;><Code[3]>
<;><MessageText[128]>
FPR operation: Reads specific message number

Input data:

'R'	1 symbol with value 'R'
'L'	1 symbol with value *'L'*
MessageNum	2 symbols for total number of messages

Output data:

'R'	1 symbol with value 'R'
'L'	1 symbol with value *'L'*
MessageNum	2 symbols for total number of messages
DateTime	Date Time parameter
Type	1 symbol for type
Code	3 symbols for code
MessageText	Up to 128 symbols for message text

zfpdef: ReadSpecificMessage()

2.8.23. Command: 4Eh / N – Read GPRS APN autehntication type

input: *<'R'><;><'G'><;><'N'>*
output: *<'R'><;><'G'><;><'N'><;><AuthenticationType[1]>*
FPR operation: Read GPRS APN authentication type

Input data:

'R'	1 symbol with value 'R'
'G'	1 symbol with value *'G'*
'N'	1 symbol with value *'N'*

Output data:

'R'	1 symbol with value 'R'
'G'	1 symbol with value *'G'*
'N'	1 symbol with value *'N'*
AuthenticationType	1 symbol with value: - '0' - None - '1' - PAP - '2' - CHAP - '3' – PAP or CHAP

zfpdef: ReadGPRS_AuthenticationType()

2.8.24. Command: 4Eh / N – Read TCP port number

input: *<'R'><;><'Z'><;><'P'>*
output: *<'R'><;><'Z'><;><'P'><;><TcpPortNumber[5]>*
FPR operation: Provides information about device's TCP port number.

Input data:

'R'	1 symbol with value 'R'
'Z'	1 symbol with value *'Z'*
'P'	1 symbol with value 'P'

Output data:

'R'	1 symbol with value 'R'
'Z'	1 symbol with value *'Z'*
'P'	1 symbol with value 'P'
TcpPortNumber	5 symbols for TCP port in format #####

zfpdef: ReadTcpPortNumber()

2.8.25. Command: 4Eh / N – Read proxy server address

input: *<'R'><;><'S'><;><'P'>*
output: *<'R'><;><'S'><;><'P'><;> <ParamLength[1..3]> <;> <Address[21]>*
FPR operation: Provides information about the proxy server address.

Input data:

'R'	1 symbol with value 'R'
'S'	1 symbol with value *'S'*
'P'	1 symbol with value 'P'

Output data:

'R'	1 symbol with value 'R'
'Z'	1 symbol with value *'Z'*
'P'	1 symbol with value 'P'
ParamLength	Up to 3 symbols for parameter length
Address	21 symbols for address

zfpdef: ReadProxyServerAddress()

2.8.26. Command: 4Eh / N – Read server reading timeout

input: *<'R'><;><'S'><;><'R'>*
output: *<'R'><;><'S'><;><'R'><;> <ReadingTimeout[4]>*
FPR operation: Provides information about timeout when reading from a server. This time is in the seconds. Minimum value: 120, maximum value: 6000.

Input data:

'R'	1 symbol with value 'R'
'S'	1 symbol with value *'S'*
'R'	1 symbol with value 'R'

Output data:

'R'	1 symbol with value 'R'
'S'	1 symbol with value *'S'*
'R'	1 symbol with value 'R'
ReadingTimeout	(Timeout) 4 symbols for timeout in format ####

zfpdef: Read_ServerReadingTimeout()

2.8.27. Command: 4Eh / N – Set TCP password

input: *<'P'><;><'Z'><;><'1'><;><PassLength[1..3]><;><Password[100]>*
output: *ACK*
FPR operation: Program device's TCP password. To apply use - SaveNetworkSettings()

Input data:

'P'	1 symbol with value 'S'
'Z'	1 symbol with value *'Z'*

'1'	1 symbol with value '1'
PassLength	(Password length) Up to 3 symbols for the password len
Password	Up to 100 symbols for the TCP password

Output data: n. a.

zfpdef: SetTCP_Password(Password_Length, Password)

2.8.28. Command: 4Eh / N – Set TCP auto start

input: *<'P'><;><'Z'><;><'2'><;><TCPAutoStart[1]>*

output: *ACK*

FPR operation: Program device's autostart TCP conection in sale/line mode. To apply use -SaveNetworkSettings()

Input data:

'P'	1 symbol with value 'S'
'Z'	1 symbol with value *'Z'*
'2'	1 symbol with value '2'
TCPAutoStart	(Auto Start) 1 symbol with value: - '0' – No - '1' - Yes

Output data: n. a.

zfpdef: SetTCP_AutoStart(TCPAutoStart)

2.8.29. Command: 4Eh / N – Set Device TCP addresses

input: *<'P'><;><'T'><;><AddressType[1]> <;><DeviceAddress[15]>*
output: *ACK*
FPR operation: Program device's network IP address, subnet mask, gateway address, DNS address. To apply use -SaveNetworkSettings()

Input data:

'P'	1 symbol with value 'P'
'T'	1 symbol with value *'T'*
AddressType	(Address type) 1 symbol with value: - '2' – IP address - '3' – Subnet Mask - '4' – Gateway address - '5' – DNS address
DeviceAddress	(Device address)15 symbols for the selected address

Output data: n. a.
zfpdef: SetDeviceTCP_Addresses(AddressType, DeviceAddress)

2.8.30. Command: 4Eh / N – Set Device TCP MAC address

input: *<'P'><;><'T'><;><'6'> <;><MACAddress[12]>*
output: *ACK*
FPR operation: Program device's MAC address . To apply use -SaveNetworkSettings()

Input data:

'P'	1 symbol with value 'P'
'T'	1 symbol with value *'T'*
'6'	1 symbol with value '6'
MACAddress	12 symbols for the MAC address

Output data: n. a.
zfpdef: SetDeviceTCP_MACAddress(mac_address)

2.8.31. Command: 4Eh / N – Set Device NTP address

input: *<'P'><;><'S'><;><'N'> <;><AddressLen[1..3]><;><NTPAddress[50]>*
output: *ACK*
FPR operation: Program device's NTP address . To apply use -SaveNetworkSettings()

Input data:

'P'	1 symbol with value 'P'
'S'	1 symbol with value *'S'*
'N'	1 symbol with value 'N'
AddressLen	(Address length)Up to 3 symbols for the address length
NTPAddress	(NTP Address)50 symbols for the device's NTP address

Output data: n. a.
zfpdef: SetDeviceNTP_Address(address_length, address)

2.8.32. Command: 4Eh / N – Set TCP DHCP enabled

input: *<'P'><;><'T'><;><'1'><;><DHCPEnabled[1]>*
output: *ACK*
FPR operation: Program device's TCP network DHCP enabled or disabled. To apply use -SaveNetworkSettings()

Input data:

'P'	1 symbol with value 'S'
'T'	1 symbols with value *'T'*
'1'	1 symbols with value '*1*'
DHCPEnabled	(Status)1 symbol with value: - '0' – Disabled - '1' – Enabled

Output data: n. a.
zfpdef: SetDHCP_Enabled(DHCP_Status)

2.8.33. Command: 4Eh / N – Set TCP WiFi network name

input: *<'P'><;><'W'><;><'N'><;><WiFiNameLength[1..3]><;><WiFiNetworkName[100]>*
output: *ACK*
FPR operation: Program device's TCP WiFi network name where it will be connected. To apply use -SaveNetworkSettings()

Input data:

'P'	1 symbol with value 'P'
'W'	1 symbol with value *'W'*
'N'	1 symbol with value 'N'
WiFiNameLength	(Name length) Up to 3 symbols for the WiFi network name len
WiFiNetworkName	(Name) Up to 100 symbols for the device's WiFi ssid network name

Output data:
zfpdef: SetWiFi_NetworkName(WiFiNetworkName_Len, WiFiNetworkName)

2.8.34. Command: 4Eh / N – Set TCP WiFi password

input: *<'P'><;><'W'><;><'P'><;><PassLength[1..3]><;><Password[100]>*
output: *ACK*
FPR operation: Program device's TCP WiFi password where it will be connected. To apply use -SaveNetworkSettings()

Input data:

'P'	1 symbol with value 'P'
'W'	1 symbol with value *'W'*
'P'	1 symbol with value 'P'
PassLength	(Password length) Up to 3 symbols for the WiFi password len
Password	Up to 100 symbols for the device's WiFi password

Output data: n. a.
zfpdef: SetWiFi_Password(Password_Len, WiFiPassword)

2.8.35. Command: 4Eh / N – Set TCP active communication module, LAN or WiFi

input: *<'P'><;><'Z'><;><'U'><;><UsedModule[1]><;>*
output: *ACK*
FPR operation: Selects the active communication module – LAN or WiFi. This option can be set only if the device has both modules at the same time. To apply use - SaveNetworkSettings()

Input data:

'P'	1 symbol with value 'P'
'Z'	1 symbols with value *'Z'*
'U'	1 symbols with value 'U'
UsedModule	(Active module) 1 symbol with value: - '1' – LAN module - '2' – WiFi module

Output data: n. a.
zfpdef: SetTCP_ActiveModule(*ActiveModule*)

2.8.36. Command: 4Eh / N – Set idle timeout

input: *<'P'><;><'Z'><;><'I'><;><IdleTimeout[4]>*
output: *ACK*
FPR operation: Program device's idle timeout setting. Set timeout for closing the connection if there is an inactivity. Maximal value – 7200, minimal value 1. 0 is for never close the connection. This option can be used only if the device has LAN or WiFi. To apply use - SaveNetworkSettings()

Input data:

'P'	1 symbol with value 'P'
'Z'	1 symbols with value *'Z'*
'I'	1 symbols with value 'I'
IdleTimeout	(Timeout) 4 symbols for Idle timeout in format ####

Output data:n. a.
zfpdef: SetIdle_Timeout(IdleTimeout)

2.8.37. Command: 4Eh / N – Set GPRS APN

input: *<'P'><;><'G'><;><'A'><;><gprsAPNlength[1..3]><;><APN[100]>*
output: *ACK*
FPR operation: Program device's GPRS APN. To apply use -SaveNetworkSettings()

Input data:

'P'	1 symbol with value 'P'
'G'	1 symbol with value *'G'*
'A'	1 symbol with value '*A*'
gprsAPNlength	(APN length) Up to 3 symbols for the APN len
APN	Up to 100 symbols for the device's GPRS APN

Output data: n. a.
zfpdef: SetGPRS_APN(APN_Len, APN)

2.8.38. Command: 4Eh / N – Set GPRS password

input: *<'P'><;><'G'><;><'P'><;><PassLength[1..3]><;><Password[100]>*
output: *ACK*
FPR operation: Program device's GPRS password. To apply use - SaveNetworkSettings()

Input data:

'P'	1 symbol with value 'P'
'G'	1 symbol with value *'G'*
'P'	1 symbol with value '*P*'
PassLength	(Password length) Up to 3 symbols for the GPRS password len
Password	Up to 100 symbols for the device's GPRS password

Output data: n. a.
zfpdef: SetGPRS_Password(Password_Len, Password)

2.8.39. Command: 4Eh / N – Set GPRS username

input: *<'P'><;><'G'><;><'U'><;><gprsUserNameLength[1..3]><;><Username[100]>*
output: *ACK*
FPR operation: Program device's GPRS user name. To apply use - SaveNetworkSettings()

Input data:

'P'	1 symbol with value 'P'
'G'	1 symbol with value *'G'*
'U'	1 symbol with value '*U*'
gprsUserNameLength	(Username length) Up to 3 symbols for the username len
Username	(Username) Up to 100 symbols for the device's GPRS username

Output data: n. a.
zfpdef: : SetGPRS_Username(Username_Len, Username)

2.8.40. Command: 4Eh / N – Set HTTPS server address

input: *<'P'><;><'S'><;><'H'><;><ParamLength[1..3]><;><Address[50]>*
output: *ACK*
FPR operation: Programs server HTTPS address.

Input data:

'R'	1 symbol with value 'R'
'S'	1 symbol with value *'S'*
'H'	1 symbol with value 'H'
ParamLength	Up to 3 symbols for parameter length
Address	50 symbols for address

Output data: n. a.
zfpdef: SetHTTPS_Address()

2.8.41. Command: 4Eh / N – Set Invoice threshold count

input: *<'P'><;><'S'><;><'I'><;><Value[1..5]>*
output: *ACK*
FPR operation: Programs invoice threshold count

Input data:

'P'	1 symbol with value 'P'
'S'	1 symbol with value *'S'*
'I'	1 symbol with value 'I'
Value	Up to 5 symbols for value

Output data: n. a.
zfpdef: SetInvoice_ThresholdCount()

2.8.42. Command: 4Eh / N – Set time threshold minutes

input: *<'P'><;><'S'><;><'T'><;><Value[1..5]>*
output: *ACK*
FPR operation: Programs time threshold minutes

Input data:

'P'	1 symbol with value 'P'
'S'	1 symbol with value *'S'*
'T'	1 symbol with value 'T'
Value	Up to 5 symbols for value

Output data: n. a.

zfpdef: SetTime_ThresholdMinutes()

2.8.43. Command: 4Eh / N – Set server communication module

input: *<'P'><;><'S'><;><'E'><;><Module[1]>*
output: *ACK*
FPR operation: Program device used to talk with the server . To apply use - SaveNetworkSettings()

Input data:

'P'	1 symbol with value 'P'
'S'	1 symbol with value *'S'*
'E'	1 symbol with value '*E*'
Module	1 symbol with value: - '0' - GSM - '1' – LAN/WiFi

Output data: n. a.

zfpdef: SetServer_UsedComModule(Module)

2.8.44. Command: 4Eh / N – Set GPRS APN autehntication type

input: *<'P'><;><'G'><;><'N'><;><AuthenticationType[1]>*
output: ACK
FPR operation: Programs GPRS APN authentication type

Input data:

'P'	1 symbol with value 'P'
'G'	1 symbol with value *'G'*
'N'	1 symbol with value '*N*'
AuthenticationType	1 symbol with value: - '0' - None - '1' - PAP - '2' - CHAP - '3' – PAP or CHAP

Output data:n. a.

zfpdef: SetGPRS_AuthenticationType()

2.8.45. Command: 4Eh / N – Set TCP port number

input: *<'P'><;><'Z'><;><'P'><;><TcpPortNumber[5]>*
output: *ACK*
FPR operation: Program TCP port of the device.

Input data:

'P'	1 symbol with value 'P'
'Z'	1 symbol with value *'Z'*
'P'	1 symbol with value 'P'
TcpPortNumber	(TCP Port) 5 symbols for TCP port number in format #####

Output data:n. a.
zfpdef: SetTcpPortNumber(TcpPortNumber)

2.8.46. Command: 4Eh / N – Set proxy server address

input: *<'P'><;><'S'><;><'P'><;> <ParamLength[1..3]> <;> <Address[21]>*
output: *ACK*
FPR operation: Program the proxy server address.

Input data:

'P'	1 symbol with value 'P'
'S'	1 symbol with value *'S'*
'P'	1 symbol with value 'P'
ParamLength	Up to 3 symbols for parameter length
Address	21 symbols for address

Output data:n. a.
zfpdef: ProgramProxyServerAddress()

2.8.47. Command: 4Eh / N – Set server reading timeout

input: *<'P'><;><'S'><;><'R'><;> <ReadingTimeout[4]>*
output: *ACK*
FPR operation: Programming timeout when reading from server. This timeout is seconds. Minimum value: 120, maximum value: 6000.

Input data:

'P'	1 symbol with value 'P'
'S'	1 symbol with value *'S'*
'R'	1 symbol with value 'R'
ReadingTimeout	(Timeout) 4 symbols for timeout in format ####

Output data:n. a.
zfpdef: ProgramServerReadingTimeout()

2.8.48. Command: 4Eh / N – Save network settings

input: *<'P'><;><'A'>*
output: *ACK*
FPR operation: After every change on Idle timeout, LAN/WiFi/GPRS usage, LAN/WiFi/TCP/GPRS password or TCP auto start networks settings this Save command needs to be execute.

Input data:

'P'	1 symbol with value 'P'
'A'	1 symbol with value 'A'

Output data:n. a.
zfpdef: SaveNetworkSettings()

2.8.49. Command: 4Eh / N – Start device LAN test

input: *<'R'><;><'T'><;><'T'>*
output: *ACK*
FPR operation: Start LAN test on the device the result

Input data:

'R'	1 symbol with value ‘R’
‘T’	1 symbol with value ‘T’
‘T’	1 symbol with value ‘T’

Output data:n. a.
zfpdef: StartLANtest()

2.8.50. Command: 4Eh / N – Start device WiFi test

input: *<'R'><;><’W’><;><’T’>*
output: *ACK*
FPR operation: Start WiFi test on the device the result

Input data:

'R'	1 symbol with value ‘R’
‘W’	1 symbol with value ‘W’
‘T’	1 symbol with value ‘T’

Output data:n. a.
zfpdef: StartWiFiTest()

2.8.51. Command: 4Eh / N – Start device GPRS test

input: *<'R'><;><’G’><;><’T’>*
output: *ACK*
FPR operation: Start GPRS test on the device the result

Input data:

'R'	1 symbol with value ‘R’
‘G’	1 symbol with value ‘G’
‘T’	1 symbol with value ‘T’

Output data:n. a.
zfpdef: StartGPRStest()

3. SOFTWARE APPLICATION REQUIREMENTS

3.1. RULES FOR USING THE COMMANDS

The commands should be used observing the following rules:

- Do not send a subsequent command prior to receiving a response of the preceding one.
- Observe the sequence of sent and received messages.
- The number of the message in every subsequent command should differ from that in the preceding one.
- Observe the two status bites of the Acknowledgment response.
- When the information received is insufficient request detailed status information – Command 20h.
- Use unpacked messages to check the standby status of the CU.
-

3.2. SAMPLE SALE TRANSACTION OF CU

The sale transaction controlled by a software application (SA) is a procedure, which consists of several commands, of which obligatory are: initiation of customer fiscal receipt (command 30h), sale registration (command 31h or 32h), payment (command 35h) and finalization of the fiscal receipt (command 38h).

Sample command sequence for issuing a customer fiscal receipt:

- fiscal receipt opening (command 30h) – contains information about the operator's number and password, the type of receipt – detailed/brief, with/without VAT
- sale registration (command 31h) – contains information about the article's name, price and VAT class as well as non-compulsory information about the quantity sold and value/percent discount/addition;
- sale registration from the article database of CU (32h) – contains the number of the article as well as non-compulsory information about the quantity sold and value/percent discount/addition;
- subtotal amount (command 33h) – contains non-compulsory parameters for, external display visualization and value/percent discount/addition of the amount accumulated;
- current receipt information (command 72h) – requires a response from the CU, which contains the current parameters of the receipt, the number of sales, the accumulated amounts by VAT classs, information about initiated or finalized payments;
- calculation and payment of VAT on VAT account (command 36h) – performs automatic calculation of VAT in the receipt and its payment on VAT account;
- payment (command 35h) – contains information about the amount due and type of payment, which may cover partially or in full the grand total amount due as well as a parameter for calculation of change due;
- fiscal receipt closure (command 38h).

4. AUXILARY GS PROTOCOL AND ESC COMMANDS

GS command	Message from the AS	FP Answer
Information	<1Dh><Info> where: **Info** – character 49h - 'I'	<'I'><Number of characters per line[2]> <;> <PLU number[5]> <;> <Departments number[3]> <;> <Operators number[2]> <;> <VAT classs number[1]> <;> <header lines number[2]> <;> <payments number[1]> <;> <logo number[2]> <;> <reserve=’00’[2]> <0Ah>

ESC/POS PROTOCOL

Used in fiscal receipt with postponed parameter XXX for reversed text.

<Start ESC Mode><flag(1byte)><End ESC Mode>